

Multimedia Retrieval - Report

Rutger Vincken¹, Bar Melinarskiy¹, and Arie Klaver¹

¹ Graduate School of Natural Sciences, Utrecht University, the Netherlands

September 16, 2025

Contents

1	Introduction	3
2	Dataset	3
3	Mathematical Notations	5
3.1	General conventions.	5
3.2	Meshes and topology.	5
3.3	Bounding boxes, centering and scaling.	5
3.4	Normalization transforms and PCA.	5
3.5	Geometric functionals.	6
3.6	Scalar shape descriptors.	6
3.7	Distribution-based descriptors and sampling.	6
3.8	Disambiguation (to match symbols used elsewhere in the text).	6
4	Technical Prerequisites	7
4.1	Framework Architecture and Design Rationale	7
5	Step 1: Read and view the data	7
5.1	Data Format Processing and Geometric Representation	8
5.2	Interactive Visualization System	8
5.2.1	Wireframe Edge Visualization	8
5.3	3D Shape Viewer Web Application	9
5.4	Hierarchical Data Organization and Metadata Extraction	9
5.5	Performance and Usability	10
5.6	3D Visualization and Display Features	10
6	Step 2: Preprocessing and cleaning	11
6.1	Step 2.1: Analyzing a single shape	12
6.2	Step 2.2: Statistics over the whole database	12
6.3	Step 2.3. Resampling outliers	16
6.4	Step 2.4: Checking the resampling	16
6.5	Removing outliers	19
6.6	Step 2.5: Normalizing shapes	23
7	Step 3: Feature extraction	26
7.1	Step 3.1: Full normalization	27
7.1.1	Pipeline Overview	27

7.1.2	Mathematical Framework	27
7.1.3	Implementation and Validation	30
7.1.4	Integration with Viewer Application	34
7.2	Step 3.2: 3D shape descriptors	35
7.2.1	Mesh preparation for feature extraction	35
7.2.2	Elementary descriptors	37
7.2.3	Shape property descriptors	51
8	Step 4: Querying	57
9	Step 5: Scalability	57
10	Step 6: Evaluation	57
	References	58

Abstract

TBD

1 Introduction

Content-based 3D shape retrieval addresses the fundamental challenge of identifying similar three-dimensional objects within large databases using geometric and topological properties rather than textual metadata. Given a query shape, such systems must rank database entries by similarity and return the most relevant matches, requiring sophisticated understanding of shape characteristics across different scales and representations.

In this work, we implement a complete 3D shape retrieval system using a collection of triangular meshes spanning diverse semantic categories and geometric complexities. Originating from the Princeton Shape Benchmark [1], the dataset includes everyday objects, vehicles, buildings, tools, and animal models, offering a broad test-suite for evaluating retrieval accuracy and computational performance.

The system was developed through six main stages: data visualization and preprocessing, geometric normalization, feature extraction, similarity metric design, spatial indexing, and performance evaluation. Each stage builds on the previous to form an end-to-end pipeline that allows querying a shape and retrieving the most similar objects from the database. The following sections describe these stages in detail and present the results of our evaluation. The full source code to our project is available via [GitHub](#).

2 Dataset

For this multimedia retrieval assignment, we utilized a comprehensive 3D shape database containing diverse geometric objects organized into semantic categories. The database comprises shapes sourced from the Princeton Shape Benchmark [1], offering a diverse range of 3D mesh objects suitable for content-based shape retrieval research. The complete data set used can be found [here](#).

The dataset comprises 2,483 3D mesh objects distributed across 69 distinct semantic categories, ranging from everyday objects like chairs, tables, and cups to more specialized items such as aircraft, musical instruments, and architectural elements. This diversity ensures comprehensive coverage of different shape characteristics, geometric complexities, and semantic domains essential for robust retrieval system evaluation. The category distribution ranges from 15 to 143 shapes per category, with an average of 36.0 shapes per category and a median of 27 shapes per category.

Each shape in the database is associated with a class label corresponding to its semantic category, derived from the original benchmark collections’ folder organization structure. These class labels serve as ground truth for evaluation purposes in our content-based shape retrieval system. The shapes are stored in OBJ format [2], a widely-used standard for 3D mesh representation, containing vertex coordinates, face connectivity information, and associated geometric metadata.

The collection, as detailed in Table 1, exhibits significant variety in geometric complexity, with shapes averaging 5,908 vertices and 12,110 faces per object. This natural variation in mesh resolution and complexity provides an excellent testbed for evaluating the robustness and effectiveness of different shape descriptors and retrieval algorithms across objects of varying geometric detail and structural characteristics. Given the inherent variability in mesh resolution and geometric representation across the collection, a systematic preprocessing and normalization pipeline was essential to ensure consistent feature extraction and reliable retrieval performance. The comprehensive details of this multi-step process are presented in Sections 6 and 7.

Table 1: 3D Shape Dataset Composition

Category	Count	Avg Vertices	Avg Faces
AircraftBuoyant	16	2,509	4,925
Apartment	23	11,924	14,838
AquaticAnimal	29	4,216	8,136
Bed	27	5,357	9,687
Bicycle	26	15,402	30,099
Biplane	45	8,978	15,099
Bird	44	2,836	5,536
Bookset	20	880	4,541
Bottle	31	3,292	6,106
BuildingNonResidential	44	3,058	5,726
Bus	20	7,732	47,260
Car	89	11,170	20,282
Cellphone	20	9,793	18,375
Chess	42	3,837	7,636
City	27	5,955	8,765
ClassicPiano	20	3,035	16,214
Computer	33	3,118	5,780
ComputerKeyboard	20	4,308	22,554
Cup	30	3,695	7,014
DeskLamp	31	4,718	8,384
DeskPhone	20	16,459	29,363
Door	44	1,151	2,171
Drum	20	2,692	15,600
Fish	20	3,898	6,529
FloorLamp	28	6,284	11,059
Glasses	20	5,161	9,562
Guitar	20	9,831	17,443
Gun	58	4,391	8,476
Hand	37	6,380	12,715
Hat	15	2,339	4,552
Helicopter	51	5,772	10,804
House	46	7,485	15,064
HumanHead	51	4,085	13,916
Humanoid	142	4,240	8,418
Insect	66	3,698	7,071
Jet	143	1,142	2,098
Knife	27	650	2,739
MilitaryVehicle	20	14,750	24,322
Monitor	20	7,513	12,996
Monoplane	20	1,644	2,921
Motorcycle	26	14,202	25,867
Mug	26	5,909	11,624
MultiSeat	45	5,096	9,451
MusicalInstrument	21	8,074	15,687
NonWheelChair	60	2,264	4,108
PianoBoard	20	2,542	14,883
PlantIndoors	65	17,213	10,505
PlantWildNonTree	27	7,688	11,221
Quadruped	57	5,240	9,504
RectangleTable	64	817	1,615
Rocket	20	7,092	4,014
RoundTable	37	1,848	3,433
Shelf	43	1,289	1,686
Ship	63	5,076	8,690
Sign	15	1,402	2,703
Skyscraper	20	25,559	25,817
Spoon	20	959	3,291
Starship	31	5,596	11,386
SubmachineGun	20	5,295	9,687
Sword	53	4,529	8,701
Tool	44	2,951	5,827
Train	30	5,273	25,320
Tree	47	15,587	13,812
Truck	27	7,176	31,413
TruckNonContainer	20	5,646	33,820
Vase	20	971	1,936
Violin	20	3,357	20,536
Wheel	17	1,192	2,218
WheelChair	20	12,465	22,059
Total	2483	5,908	12,110

3 Mathematical Notations

3.1 General conventions.

- Bold lowercase letters ($\mathbf{x}, \mathbf{y}, \mathbf{v}$) denote vectors in $\mathbb{R}^3$; uppercase Greek (e.g., Σ) denote matrices.
- $\|\cdot\|_2$: Euclidean norm; $\cdot$: dot product; $\times$: cross product.
- For a set S , $|S|$ is its cardinality; for a scalar a , $|a|$ is absolute value (context disambiguates).
- $\text{sign}(\cdot)$: sign function; $\odot$: element-wise (Hadamard) product.

3.2 Meshes and topology.

- $M = (V, F)$: a triangular mesh.
- $V = \{\mathbf{v}_1, \dots, \mathbf{v}_n\} \subset \mathbb{R}^3$: vertex set; $n = |V|$.
- $F = \{f_1, \dots, f_m\}$: face set with $f_j = (i, j, k)$ an ordered triple of vertex indices; $m = |F|$.
- E : set of unique undirected mesh edges extracted from F .
- T : set of triangles (we use $T \equiv F$ when summing over triangles).
- $\mathbf{n}_f$: unit normal of face f ; $\mathbf{n}_i$: vertex normal at $\mathbf{v}_i$ (area-weighted average of incident face normals).
- $\tau(f_j)$: polygon type of face f_j (triangle, etc.); $\mathcal{T}(M) = \{\tau(f_j) \mid f_j \in F\}$.
- $c \in \mathcal{C}_{\text{cls}}$: semantic class label; $\mathcal{C}_{\text{cls}}$ is the set of classes.
- $\mathcal{C}_{\text{cmp}}(M)$: set of connected face components of M (used in the signed-volume sum). *Note.* When $\mathcal{C}(M)$ appears in formulas, it refers to $\mathcal{C}_{\text{cmp}}(M)$.

3.3 Bounding boxes, centering and scaling.

- $B(M) = [\mathbf{v}_{\min}, \mathbf{v}_{\max}]$: axis-aligned bounding box (AABB), with $\mathbf{v}_{\min} = (\min_i v_i^x, \min_i v_i^y, \min_i v_i^z)$, $\mathbf{v}_{\max} = (\max_i v_i^x, \max_i v_i^y, \max_i v_i^z)$.
- $\mathbf{s} = (s_x, s_y, s_z) = \mathbf{v}_{\max} - \mathbf{v}_{\min}$; $s_{\max} = \max(s_x, s_y, s_z)$; uniform scale $\alpha = 1/s_{\max}$.
- $\mathbf{b}(M)$: mass barycenter (area-weighted triangle centroid average). For triangle $t = (a, b, c)$, $A(t) = \frac{1}{2} \|(\mathbf{v}_b - \mathbf{v}_a) \times (\mathbf{v}_c - \mathbf{v}_a)\|$, $c(t) = \frac{1}{3}(\mathbf{v}_a + \mathbf{v}_b + \mathbf{v}_c)$, and $\mathbf{b}(M) = (\sum_{t \in T} A(t)c(t)) / (\sum_{t \in T} A(t))$.
- Tolerances: $\epsilon_{\text{area}} > 0$ (degenerate-area guard), $\epsilon_{\text{recenter}} > 0$ (recentering threshold).

3.4 Normalization transforms and PCA.

- $v_i^{(0)} = \mathbf{v}_i$ (original). After step $k \in \{1, 2, 3, 4\}$: $v_i^{(k)} = (T_k \circ \dots \circ T_1)(\mathbf{v}_i)$:
 - (1) T_1 (translation): center at $\mathbf{b}(M)$.
 - (2) T_2 (alignment): PCA alignment to principal axes.
 - (3) T_3 (flipping): axis-sign disambiguation via signed moments.
 - (4) T_4 (scaling): fit to unit cube (max side length = 1).
- PCA on centered vertices: covariance Σ , eigenpairs $(\lambda_i, \mathbf{u}_i)$, sorted $\lambda_1 \geq \lambda_2 \geq \lambda_3 \geq 0$.
- Right-handed orthonormal basis: $\mathbf{e}_1 = \mathbf{u}_1 / \|\mathbf{u}_1\|$, $\mathbf{e}_2 = \mathbf{u}_2 / \|\mathbf{u}_2\|$, $\mathbf{e}_3 = \mathbf{e}_1 \times \mathbf{e}_2$. Aligned coordinates: $v_j^{(2)} = [\mathbf{e}_1^\top v_j^{(1)}, \mathbf{e}_2^\top v_j^{(1)}, \mathbf{e}_3^\top v_j^{(1)}]^\top$.
- Flipping moments: for triangle centroid $\mathbf{C}_t$, $f_i = \sum_{t \in T} \text{sign}(C_{t,i}) C_{t,i}^2$ ($i = 0, 1, 2$). Apply $v^{(3)} = v^{(2)} \odot [\text{sign}(f_0), \text{sign}(f_1), \text{sign}(f_2)]^\top$.

- Axis extents after flipping: $d_i = \max_j v_{j,i}^{(3)} - \min_j v_{j,i}^{(3)}$, $d_{\max} = \max(d_0, d_1, d_2)$, scale $v^{(4)} = v^{(3)} / d_{\max}$.

3.5 Geometric functionals.

- $\text{Area}(M) = A(M) = \frac{1}{2} \sum_{(i,j,k) \in F} \|(\mathbf{v}_j - \mathbf{v}_i) \times (\mathbf{v}_k - \mathbf{v}_i)\|$.
- $\text{Vol}(M)$ (a.k.a. $V(M)$ in formulas): per-component signed volume summed in absolute value, $\text{Vol}(M) = \sum_{c \in \mathcal{C}_{\text{cmp}}(M)} \left| \frac{1}{6} \sum_{(i,j,k) \in F_c} \mathbf{v}_i \cdot ((\mathbf{v}_j - \mathbf{v}_i) \times (\mathbf{v}_k - \mathbf{v}_i)) \right|$.
- M^{hull} : convex hull of M .
- Oriented bounding box (OBB): side lengths (w, h, d) with $V_{\text{OBB}}(M) = w h d$.
- $D(M) = \max_{\mathbf{p}, \mathbf{q} \in V} \|\mathbf{p} - \mathbf{q}\|_2$: (graph-independent) geometric diameter.

3.6 Scalar shape descriptors.

- **Surface-to-Volume Ratio**: $\text{SVR}(M) = \frac{A(M)}{\text{Vol}(M)}$. This simple but informative ratio grows with increasing surface complexity and decreases for smoother, more compact shapes. It complements the compactness measure by remaining in physical units rather than being normalized to a sphere.
- **Sphericity**: $\Psi(M) = \frac{\pi^{1/3} (6 \text{Vol}(M))^{2/3}}{A(M)}$. A perfect sphere yields $\Psi = 1$; more elongated or irregular objects have smaller values. This descriptor provides an intuitive measure of how closely a 3D object approximates a sphere.
- **Compactness** (denoted here by $\text{Comp}(M)$; equals $\mathcal{C}(M)$ in Section 7): $\text{Comp}(M) = \frac{A(M)^3}{36\pi \text{Vol}(M)^2}$.
- **Rectangularity**: $\mathcal{R}(M) = \frac{\text{Vol}(M)}{V_{\text{OBB}}(M)} \in (0, 1]$.
- **Convexity**: $\mathcal{CX}(M) = \frac{\text{Vol}(M)}{\text{Vol}(M^{\text{hull}})} \in (0, 1]$.
- **Eccentricity**: with PCA eigenvalues $\lambda_1 \geq \lambda_2 \geq \lambda_3$, $\varepsilon(M) = \lambda_1 / \lambda_3$.

3.7 Distribution-based descriptors and sampling.

- Default sampling: $S = 100,000$ i.i.d. samples; histogram bins $H = 20$; histograms are *L1-normalized*.
- **A3** (angle): for $(\mathbf{x}, \mathbf{y}, \mathbf{z}) \in V^3$, $\theta = \arccos \frac{(\mathbf{y}-\mathbf{x}) \cdot (\mathbf{z}-\mathbf{x})}{\|\mathbf{y}-\mathbf{x}\| \|\mathbf{z}-\mathbf{x}\|}$. Histogram limits use the empirical 99th percentile plus 10% margin.
- **D1** (barycenter distance): $d_1 = \|\mathbf{x} - \mathbf{b}(M)\|_2$ for $\mathbf{x} \in V$.
- **D2** (pairwise distance): $d_2 = \|\mathbf{x} - \mathbf{y}\|_2$ for $(\mathbf{x}, \mathbf{y}) \in V^2$.
- **D3** (triangle area, square-rooted): for $(\mathbf{x}, \mathbf{y}, \mathbf{z}) \in V^3$, $A_{\Delta} = \frac{1}{2} \|(\mathbf{y} - \mathbf{x}) \times (\mathbf{z} - \mathbf{x})\|$; histogram uses $\sqrt{A_{\Delta}}$.
- **D4** (tetrahedron volume, cube-rooted): for $(\mathbf{w}, \mathbf{x}, \mathbf{y}, \mathbf{z}) \in V^4$, $V_{\text{tet}} = \frac{1}{6} |(\mathbf{x} - \mathbf{w}) \cdot ((\mathbf{y} - \mathbf{w}) \times (\mathbf{z} - \mathbf{w}))|$; histogram uses $\sqrt[3]{V_{\text{tet}}}$.
- $P_{99}[\cdot]$: empirical 99th percentile used to set descriptor-specific histogram maxima (then expanded by 10%).

3.8 Disambiguation (to match symbols used elsewhere in the text).

- V denotes the *vertex set*. When you see $V(M)$ in equations for volume, read it as $\text{Vol}(M)$.

- $\mathcal{C}$ is overloaded in the manuscript: $\mathcal{C}_{\text{cls}}$ (class set), $\mathcal{C}_{\text{cmp}}(M)$ (connected components), and $\text{Comp}(M)$ (compactness; written as $\mathcal{C}(M)$ in Sec. 7). The above table makes each meaning explicit to avoid ambiguity.

4 Technical Prerequisites

The following technical prerequisites were required for the development and execution of our 3D shape retrieval system:

- **Programming Languages:** Python 3.10
- **Third-party Libraries:**
 - ☐ Dash – web application framework
 - ☐ Plotly – interactive 3D visualization
 - ☐ Pandas – data manipulation and analysis
 - ☐ NumPy – numerical computations
 - ☐ Open3D - an open-source library that supports rapid development of software that deals with 3D data
 - ☐ Scikit-learn - an open source machine learning library that supports supervised and unsupervised learning
- **Development Tools and Frameworks:**
 - ☐ Visual Studio Code – primary IDE
 - ☐ Git – version control
 - ☐ Anaconda – environment management

4.1 Framework Architecture and Design Rationale

The viewer app was developed as a web-based application, using Dash [3] to manage reactive interactions and Plotly [4] to provide efficient 3D visualization capabilities. The selection of this technology stack was guided by three requirements:

1. Real-time interactive exploration of the large mesh dataset.
2. Cross-platform accessibility without the need for specialized 3D software.
3. Seamless integration with Python-based scientific computing tools for efficient data processing, such as Pandas [5] and NumPy [6].

The framework architecture follows a model–view–controller paradigm [7], which separates data processing (model), user interface (view), and interaction logic (controller). This separation improves maintainability and supports extensibility for the integration of subsequent retrieval components.

5 Step 1: Read and view the data

Effective retrieval requires a clear understanding of the dataset at hand. To address this, our system begins with the development of an exploration and visualization framework tailored to 3D meshes. The following subsections provide a detailed breakdown of the application’s implementation.

The purpose of this stage is to establish a comprehensive environment for systematic analysis of the mesh datasets. Interactive visualization is a central component in multimedia retrieval, as it enables researchers to assess data distributions, identify potential preprocessing challenges, and validate algorithmic decisions.

5.1 Data Format Processing and Geometric Representation

Our system processes triangular mesh data stored in the Wavefront OBJ format, which has become the de facto standard for 3D model interchange due to its simplicity and widespread software support [2]. The OBJ format represents 3D objects through two primary components: vertex coordinates defining spatial geometry and face definitions specifying triangular connectivity.

We implemented a custom `OBJParser` class that addresses several technical challenges inherent in OBJ processing:

- **Index Convention Translation:** OBJ files use 1-based indexing while NumPy arrays require 0-based
- **Polygon Triangulation:** Automatic conversion of quad faces to triangles through diagonal splitting, ensuring consistent triangular mesh representation required for downstream processing
- **Robust Parsing:** Error handling for malformed geometry definitions and missing vertex references

The parser extracts vertex coordinates $V = \{v_1, v_2, \dots, v_n\}$ where $v_i \in \mathbb{R}^3$ and face connectivity $F = \{f_1, f_2, \dots, f_m\}$ where each face f_j contains three vertex indices, forming the fundamental mesh representation $M = (V, F)$ used throughout our retrieval pipeline.

5.2 Interactive Visualization System

The visualization component leverages Plotly’s WebGL-based 3D rendering engine, which provides hardware-accelerated graphics suitable for real-time interaction with complex meshes [8]. Our implementation incorporates several advanced rendering techniques:

To visualize 3D shapes in our web application, we use the `textitMesh3d` object from the Plotly Graph Objects library [9]. This tool allows us to render interactive 3D meshes directly in the browser, providing smooth rotation, zooming, and lighting effects. By leveraging Plotly’s capabilities, we can display complex surfaces and wireframes in a user-friendly way.

5.2.1 Wireframe Edge Visualization

Our optional wireframe overlay uses an efficient edge extraction method that avoids duplicate edges by storing them in a set. For each face f in the set of faces F , we iterate over its edges. Each edge is represented by a pair of vertex indices, sorted to ensure consistency. We add each sorted edge to the set E , which automatically removes duplicates.

This set-based approach guarantees $O(F)$ complexity for edge extraction, where F is the number of faces. The resulting set contains E unique edges. This significantly reduces rendering overhead compared to naive enumeration of all face edges [10, pp. 17–19].

Algorithm 1: Efficient Wireframe Edge Extraction

Input: Set of faces F (each face is a list of vertex indices)

Output: Set of unique edges E

```
1  $E \leftarrow \emptyset$ ; // Initialize empty edge set
2 foreach  $face\ f \in F$  do
3   for  $i \leftarrow 0$  to  $|f| - 1$  do
4      $v_1 \leftarrow f[i]$ ;
5      $v_2 \leftarrow f[(i + 1) \bmod |f|]$ ;
6      $edge \leftarrow \text{sort}(v_1, v_2)$ ; // Sort vertex indices for consistency
7      $E \leftarrow E \cup \{edge\}$ ; // Add edge to set (deduplication)
8   end
9 end
10 return  $E$ 
```

5.3 3D Shape Viewer Web Application

Our interactive web application, shown in Figure 1, enables users to explore 3D models directly in their browser. The figure displays an example visualization of a bird mesh. Below, we describe the main components and design choices that support usability, performance, and data quality.

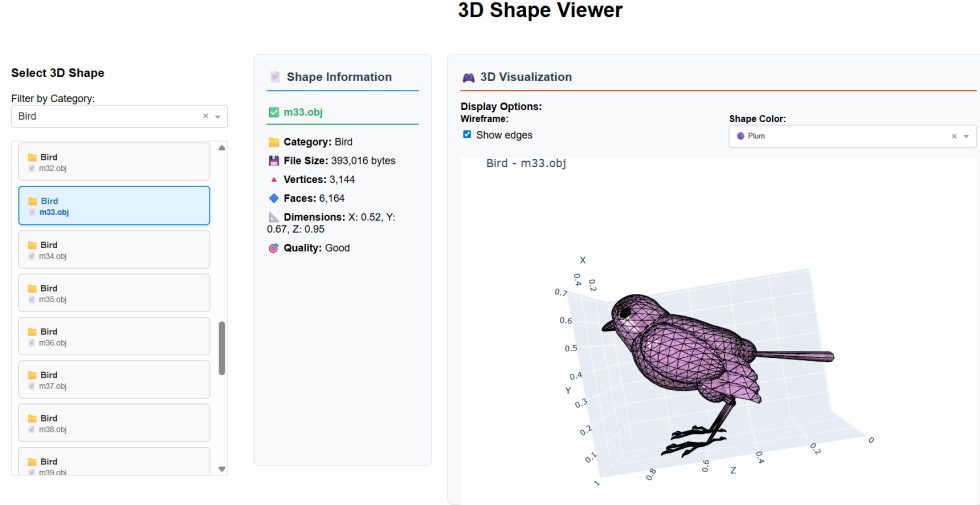


Figure 1: **Initial version of interactive user interface for 3D shape retrieval.** Users can filter the object list by category, such as “Bird,” or scroll through the complete list. Selecting a model reveals detailed information, including file size, vertices, faces, dimensions, and quality, along with an interactive 3D visualization. Display options such as wireframe edges and customizable colors support both structural inspection and visual comparison of shapes.

5.4 Hierarchical Data Organization and Metadata Extraction

To facilitate systematic exploration and evaluation, we utilized the organization of the 3D models into a hierarchical directory structure by category. The application automatically scans these folders, identifies available models, and extracts key metadata such as vertex and face counts, file size, and bounding box dimensions. This metadata is presented to users for quick assessment and is also used to label models as high or low resolution, supporting both user selection and algorithmic preprocessing. As shown in Figure 2, users can filter models by category to quickly focus on the objects they wish to view.

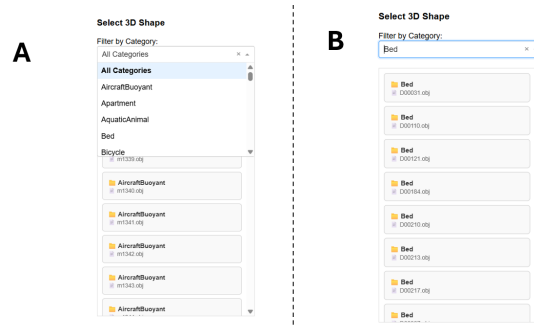


Figure 2: **Filter by category.** (A) The user can filter by a specific category or display objects from all categories. (B) For example, we filtered on all objects of beds.

5.5 Performance and Usability

To ensure a smooth user experience, our application is optimized for performance and reliability. Model files are loaded and parsed for display only when selected, which keeps memory usage low and startup times fast. Client-side handling of simple interactions, such as file selection highlighting, provides immediate feedback. Robust error handling further enhances usability by notifying users of any issues during file loading.

5.6 3D Visualization and Display Features

The viewer enables users to interactively explore 3D models using Plotly’s `Mesh3d` object. Users can rotate, zoom, and pan the models, as well as customize display options such as mesh color, wireframe edges, and smooth shading. The smooth shading feature allows users to toggle between flat and smooth surface rendering for enhanced geometric inspection (see Figure 4). These features support both visual comparison and structural inspection, making it easier to understand the geometry and quality of each shape. These capabilities are demonstrated in Figures 3, 5, and 4.

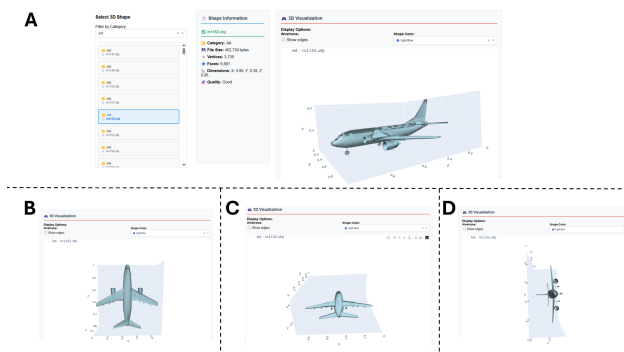


Figure 3: **Rotation demonstration.** (A) Example display of a plane object and its metadata; (B–D) the same object shown at different rotation angles.

To enable the user to control the shape’s shading, we utilized the *flatshading* parameter of Plotly’s `Mesh3d` object. This parameter controls whether normal smoothing is applied to the mesh: when enabled, the mesh is rendered with flat, angular faces for a low-poly appearance; when disabled, smooth shading is used, resulting in a visually softer surface with interpolated lighting across vertices. An example of this feature can be seen in Figure 4.

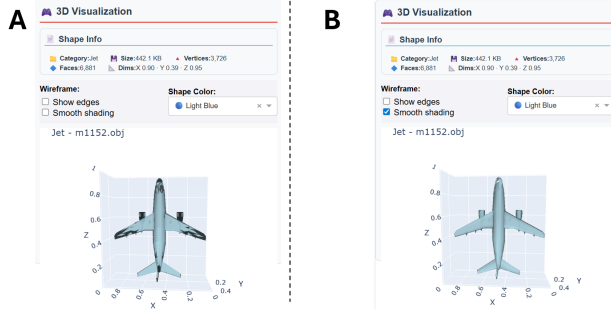


Figure 4: **Smooth shading demonstration.** (A) Example display of a plane object and its metadata using Plotly’s default flat shading, where each face is rendered with a uniform color and sharp edges are visible. (B) The same object shown with smooth shading enabled, resulting in a visually smoother surface appearance as lighting is interpolated across vertices. This feature allows users to toggle between flat and smooth shading in the viewer for enhanced geometric inspection and comparison.

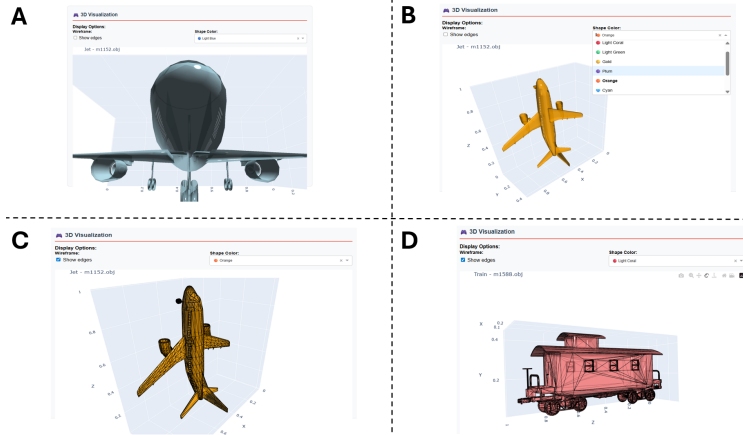


Figure 5: **Display features demonstration.** (A) Zooming in on the same plane object from Figure 3. (B) Changing the object’s color from blue to orange. (C) Showing the plane’s edges by enabling the ”Show edges” checkbox above the viewer. (D) Displaying a pink train object with its edges visible, illustrating both color change and edge visualization.

This application streamlines the process of inspecting and comparing 3D models, providing both visual feedback and essential metadata to support further analysis and retrieval experiments.

6 Step 2: Preprocessing and cleaning

The dataset contains a large variety of 3D objects that differ significantly in complexity, resolution and overall quality. Before features can be reliably extracted and used for retrieval, it is essential to ensure that all shapes satisfy certain quality requirements. Poorly sampled meshes, inconsistent scaling, or misaligned objects may bias the descriptors extracted later on, which in turn reduces the effectiveness and fairness of the retrieval system.

In this section, we analyze the structural characteristics of all shapes in the database and identify common irregularities such as extreme outliers in the number of vertices and faces, as well as variations in scale

and positioning. By collecting global statistics over the dataset, we obtain a better understanding of its variability and can determine which objects require resampling or refinement.

Next, we apply a set of preprocessing operations to improve dataset consistency. These operations include mesh simplification for overly large shapes, refinement for undersampled ones, and normalization to standardize the position and scale of each object. The resulting database forms a more balanced and homogeneous foundation for the subsequent feature extraction stage, ensuring that differences in descriptors reflect meaningful shape characteristics rather than artifacts of the raw input data.

6.1 Step 2.1: Analyzing a single shape

Before starting the preprocessing and cleaning of the dataset, we first obtain an overview of the initial variability of the objects. For this purpose, we implemented a simple analysis tool that reports, for each shape $M = (V, F)$, the following properties:

- The semantic class label $c \in \mathcal{C}$ of the shape.
- The number of vertices $|V|$ and the number of faces $|F|$.
- The set of face types $\mathcal{T}(M)$, which indicates whether the shape consists of only triangles, only quadrilaterals, or a mixture of polygon types.
- The axis-aligned bounding box $B(M) = [\mathbf{v}_{\min}, \mathbf{v}_{\max}]$, defined by the minimum and maximum coordinates among all vertices.

This information provides a concise summary of the raw dataset characteristics and highlights potential sources of variability that may need to be addressed in subsequent preprocessing steps.

The tool operates by iterating over all files in the dataset in `.obj` format. In this format, each line encodes either a vertex position (`v`), a vertex normal (`vn`), or a face (`f`). During parsing, each vertex is added to the set V , and each face to the set F , while ignoring additional attributes such as normals and texture coordinates. Once all lines have been processed, we directly obtain $|V|$ and $|F|$. The bounding box $B(M)$ is computed by taking the coordinate-wise minima and maxima of the vertices. Face types are determined by checking the number of vertices in each face, distinguishing between triangles, quadrilaterals, and higher-order polygons.

The extracted statistics are written to a comma-separated values (CSV) file, producing one record per shape. In the next step, we will aggregate these records to obtain statistics over the entire dataset.

6.2 Step 2.2: Statistics over the whole database

Having obtained the individual characteristics of each object, we now analyze their distribution across the entire dataset. We start by examining the distribution of shapes across the different classes. The resulting histogram is shown in Figure 6.

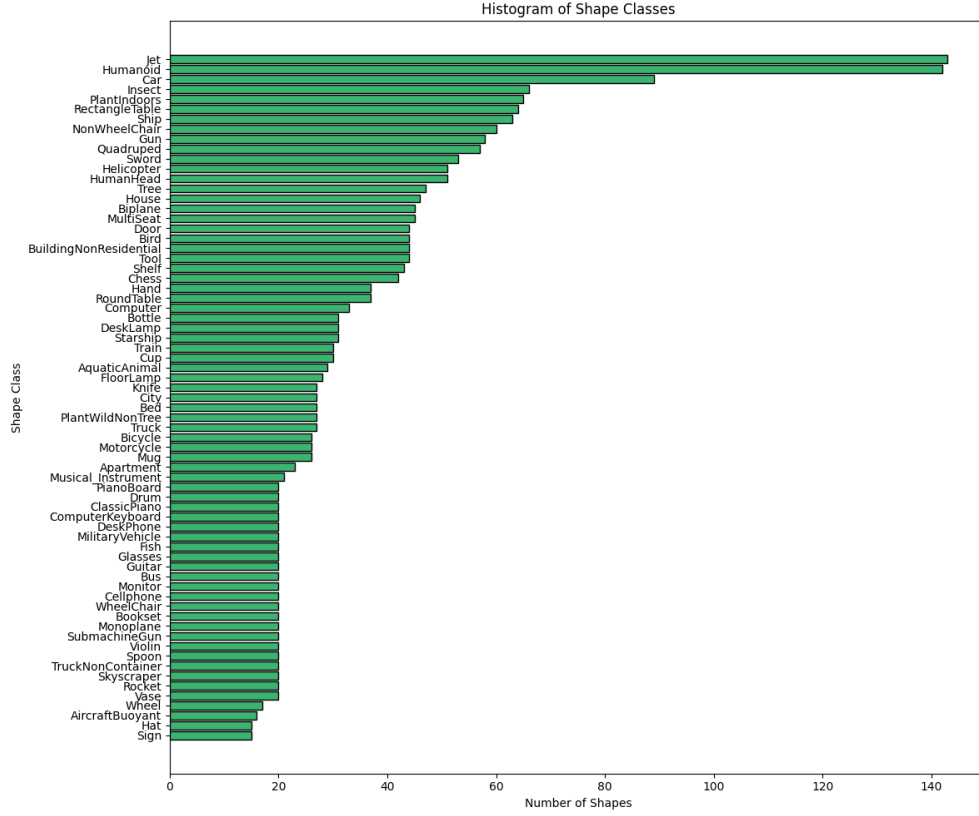


Figure 6: **Distribution of shape classes.** The histogram shows the number of shapes per class in the dataset. The vertical axis indicates the shape classes, while the horizontal axis represents the number of shapes in each class.

From the histogram, we observe that most classes contain between 20 and 50 shapes. There are a few outliers with either fewer shapes or significantly more. Notably, the *jet* and *humanoid* classes contain over 140 shapes each, representing the largest class sizes in the dataset.

Now that we have an overview of the distribution of shapes across classes, we next examine the vertex counts of the shapes. To visualize this, we plot a histogram of the vertex counts, shown in Figure 7. The histogram spans the range from 0 to 100,000 vertices and is divided into 30 uniform bins, each covering approximately 3,333 vertices.

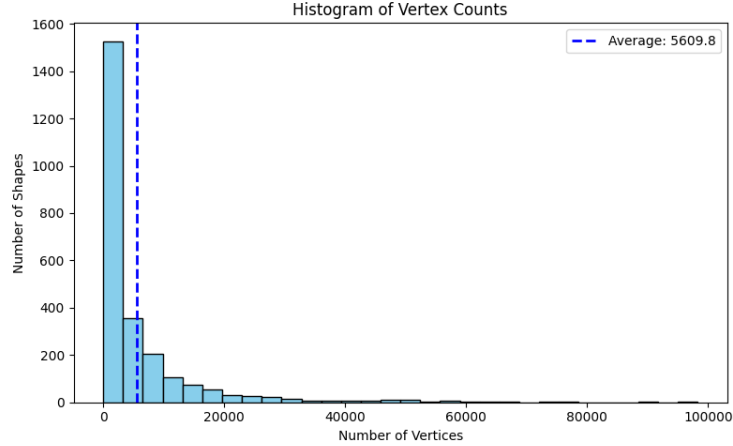


Figure 7: **Distribution of vertex counts.** Histogram of the number of vertices per shape. The horizontal axis shows vertex count (in bins of $\sim 3,333$ vertices) and the vertical axis shows the number of shapes in each bin. The dotted line indicates the mean vertex count.

We observe that most shapes fall into the lower bins, with vertex counts below 3,333. The mean vertex count of 5610 lies outside this first bin, indicating a skew towards higher counts. In addition, there are several clear outliers, with vertex counts exceeding 90,000.

The average shape, based on vertex count, is `D00926.obj`, shown in Figure 8. We also show several outliers.

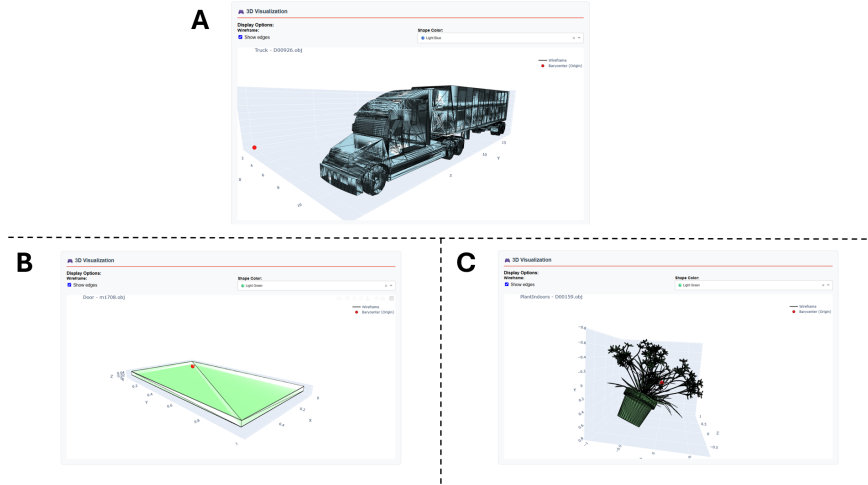


Figure 8: **Average shape and outliers for number of vertices.** (A) The average shape in the dataset has 5,610 vertices. The shape closest to this average is object `D00926.obj` of the *Truck* class, with 5,602 vertices. (B) The shape with the lowest number of vertices is *Door* object `m1708.obj`, with 16 vertices. (C) The shape with the highest number of vertices is *PlantIndoors* object `D00159.obj`, with 98,256 vertices.

We also examine the face count of the shapes. We again visualize this by plotting a histogram, this time of the face counts, as shown in Figure 9. The histograms spans the range from 0 to 140,000 faces and is divided into 30 uniform bins, each covering approximately 4,667 faces.

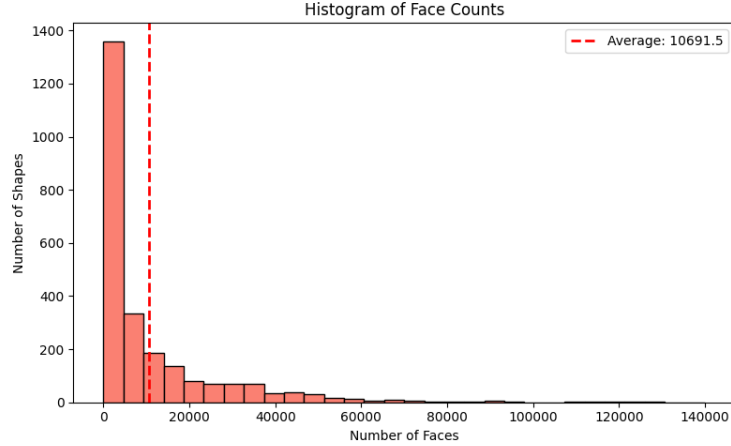


Figure 9: **Distribution of Face counts.** Histogram of the number of faces per shape. The horizontal axis shows face count (in bins of $\sim 4,667$ vertices) and the vertical axis shows the number of shapes in each bin. The dotted line indicates the mean face count.

We observe a similar distribution as with the vertex count, with a large spike in the lowest bin and a skew towards higher counts. The average however, lies higher then for the vertex count, at 10,692 faces. We also see similar outliers, this time ranging up to 130,000.

The average shape, based on the face count, is `m1120.obj`, shown in Figure 10. We also show several outliers.

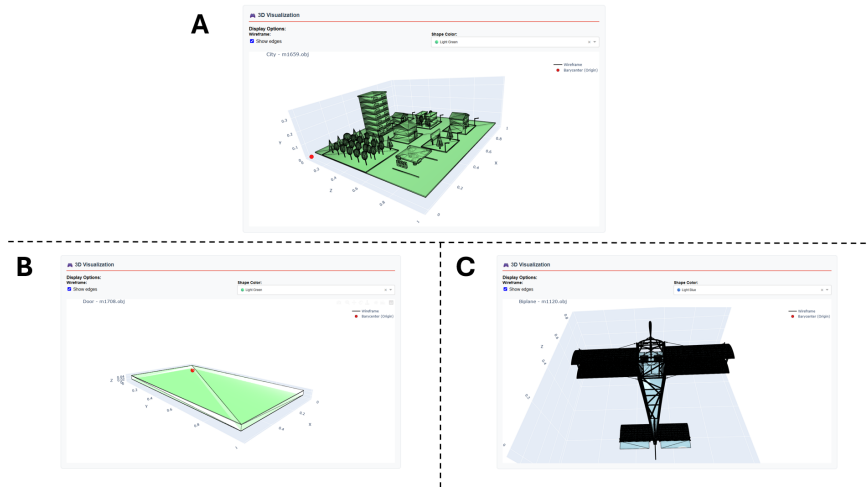


Figure 10: **Average shape and outliers for number of faces.** (A) The average shape in the dataset has 10,692 faces. The shape closest to this average is object `m1659.obj` of the *City* class, with exactly 10,692 faces. (B) The shape with the lowest number of faces is *Door* object `m1708.obj`, with 16 faces. (C) The shape with the highest number of faces is *Biplane* object `m1120.obj`, with 129,881 faces.

We also inspected the face types of the objects in the dataset, classifying them as consisting of only triangles, only quadrilaterals, or a mixture of polygon types. We found that all objects consist solely of triangular faces, so no further processing was required in this regard.

6.3 Step 2.3. Resampling outliers

From the obtained statistics it has become clear that the dataset contains a wide variety of shapes. To ensure fair and unbiased feature extraction and object retrieval, it is necessary to reduce the influence of extreme outliers in terms of vertex count. For this reason, we apply a remeshing step that regularizes the mesh resolution across the dataset. This ensures that all shapes are represented at a comparable level of detail, making subsequent analysis and retrieval more consistent and reliable.

For the remeshing step, it is necessary to specify a target resolution in terms of vertex count. We chose a nominal target of 7,500 vertices, which offers a good compromise between preserving geometric detail and maintaining manageable processing times. Because it is often impractical to achieve exactly this number of vertices, we allow a tolerance range of 5,000 to 10,000 vertices. Remeshing therefore stops once the mesh falls within this range, ensuring both consistency across the dataset and robustness of the preprocessing pipeline.

The remeshing procedure was implemented using **Open3D** [11] (version ≥ 0.18), an open-source library for 3D data processing. Open3D provides efficient tools for handling triangle meshes, including vertex and triangle access, mesh cleaning, simplification, and subdivision. Its functions for *quadric-based decimation* and *midpoint subdivision* were used to adjust mesh resolution while preserving the shape of the geometry. Additionally, **NumPy** [6] was employed for numerical computations, nearest-neighbor searches, and vertex reprojection.

The goal of the remeshing step is to reduce the variation in mesh resolution across the dataset and limit the influence of outlier meshes. The algorithm works as follows:

1. **Vertex Count Check:** Each mesh is first checked against the target vertex range of 5,000 to 10,000 vertices (nominal target: 7,500). Meshes that are already within this range are copied without modification.
2. **Upsampling:** Meshes with fewer vertices than the minimum threshold are increased in resolution using a *feature-preserving midpoint subdivision*. New vertices are projected toward local planes or neighborhood centroids to maintain surface features and prevent excessive smoothing. This process is repeated in multiple passes until the mesh reaches the desired resolution.
3. **Downsampling:** Meshes exceeding the maximum vertex threshold are simplified using *quadric-based decimation*. The reduction is applied iteratively in multiple passes, with the reduction ratio adapted depending on how far the mesh is from the target. If the decimation stalls, a more aggressive pass is applied to ensure the mesh falls within the target range.
4. **Feature Preservation:** Sharp edges and important geometric features are identified based on dihedral angles between adjacent faces. During subdivision and decimation, these features guide the placement of new vertices and the removal of faces to retain the overall shape. Planar regions are preserved by blending vertex positions toward fitted local planes when possible.
5. **Fallback and Robustness:** In rare cases where standard upsampling fails, a single subdivision pass followed by light decimation is used. Mesh cleaning operations, such as removal of degenerate triangles and duplicated vertices, are applied throughout to maintain mesh quality.

This approach ensures that all meshes in the dataset have a comparable number of vertices while keeping their geometric features intact, which facilitates consistent feature extraction and object retrieval.

6.4 Step 2.4: Checking the resampling

We now evaluate the effect of the remeshing algorithm on the dataset. Again, a practical way to assess this is by examining the distribution of vertex counts across all meshes. Figure 11 shows the resulting histogram. The plot indicates that the variance in vertex counts has been substantially reduced compared to the original data. The majority of meshes now fall within the intended target range, and extreme outliers have largely been removed or adjusted.

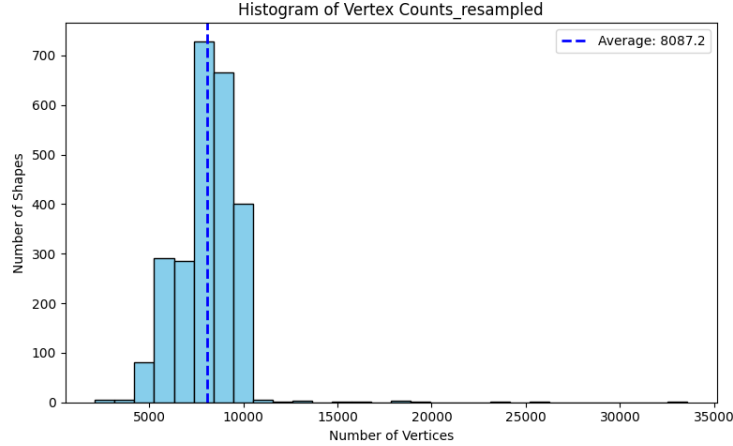


Figure 11: **Distribution of vertex counts.** Histogram of the number of vertices per shape. The horizontal axis shows vertex count (bins of approximately 3,333 vertices), and the vertical axis shows the number of shapes in each bin. The dotted line indicates the mean vertex count.

In addition, we have generated figures illustrating both the compliance rate and the distribution of meshes across all size categories: very small, small, target range, large, and very large. These comparisons, before and after resampling, are shown in Figure 12. The compliance rate has improved considerably, from 14.8% to 97.5%, leading to only 63 out of the 2,483 objects not falling within the target vertex count range. While some meshes remain outside the target range, they are generally only slightly above or below the desired limits. Before resampling, the majority of meshes fell into the very small or small categories, with only a few within the target range and some extreme outliers in the very large category. The resampling step has successfully shifted most meshes into the target range, reducing the prevalence of undersized meshes and eliminating extreme outliers.

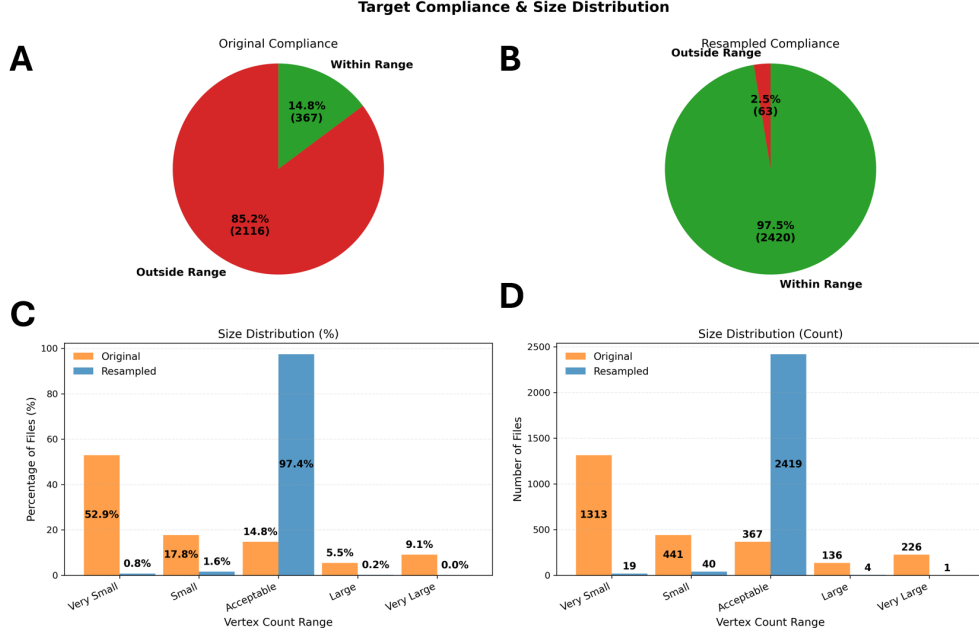


Figure 12: **Target compliance and size distribution.** (A) Pie chart showing the proportion of meshes that fall within the acceptable target range of 5,000–10,000 vertices before resampling. (B) Pie chart showing the proportion of meshes within the acceptable target range after resampling. (C) Bar chart showing the percentage distribution of meshes across all size categories before and after resampling: very small (0–2,000 vertices), small (2,000–5,000), acceptable (5,000–10,000), large (10,000–15,000), and very large (15,000+). (D) Bar chart showing the count distribution of meshes across all size categories before and after resampling.

Although the resampling pipeline substantially improved uniformity in vertex counts, a small number of meshes still fall outside of the target range of 5,000–10,000 vertices. This outcome can be explained by the nature of the subdivision and simplification algorithms used. Subdivision increases the number of vertices in large multiplicative steps (approximately by a factor of four for each pass). As a result, meshes that start well below the lower bound often overshoot into the higher range. Simplification is then applied to reduce these meshes, but the algorithm is constrained by geometry preservation: it avoids collapsing important edges or thin structures, and therefore does not always converge exactly within the target window. Similarly, meshes that were already close to the lower bound are sometimes left unchanged, as subdivision would push them unnecessarily far above the acceptable range.

Further attempts to enforce exact compliance would require mesh-specific adjustments, which would be computationally expensive and risk degrading geometric fidelity. The current procedure achieves compliance for approximately 97% of the dataset, which we consider sufficient for our purposes. The small number of outliers (63) does not meaningfully affect downstream tasks, and avoiding excessive processing helps maintain higher shape quality overall. We will further explore the outliers mentioned above in Sub-Section 6.5.

It should also be noted that resampling occasionally introduces some surface artifacts. For instance, some meshes show a rippling effect along curved surfaces, such as the body of a bottle, or slight deformations in thin structures, such as the wings of an airplane no longer being perfectly straight (see Figures 13 and 14). These cases illustrate the trade-off between vertex count normalization and preservation of fine-grained geometry. While not ideal, such artifacts are limited in scope and do not compromise the overall usability of the dataset.

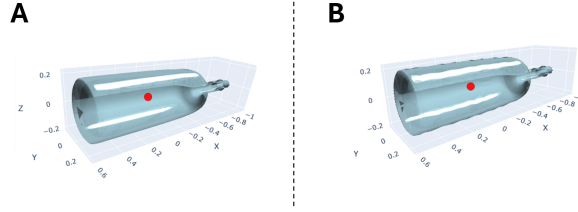


Figure 13: **Example of rippling effect after remeshing process.** (A) The original *Bottle* object (D00347.obj), consisting of 2,426 vertices and 4,438 faces. (B) The resampled *Bottle* object, now consisting of 8,624 vertices and 17,184 faces. A slight rippling effect can be observed along the edge of the bottle as a result of the remeshing process.

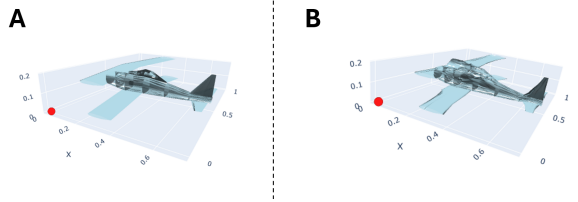


Figure 14: **Example of deformation of straight surfaces after remeshing process.** (A) The original *Biplane* object (m1131.obj), consisting of 482 vertices and 768 faces. (B) The resampled *Biplane* object, now consisting of 6,530 vertices and 12,288 faces. The previously straight wings of the airplane now have a slight deformation as a result of the remeshing process.

6.5 Removing outliers

To further check our meshing approach, we zoomed in on a few cases from the 63 shapes that were outside the compiled vertices range. Upon closer inspection, we found that most outliers shared a common characteristic: they were poorly sampled in their original form. The meshing process - which for these low-vertex-count cases involved cleaning followed by upscaling - created holes and detached parts in these problematic cases. For instance, in the jet example (Figure 15), the meshing process both introduced structural defects and overshot the target vertex range (5,000–10,000), reaching 15,794 vertices. In contrast, the bird example (Figure 16) demonstrates a case where the approach implemented an early stop at 1,693 vertices to prevent further distortion of the shape. Interestingly, both the jet and the bird exhibit the most severe problems in their wing structures, which were poorly sampled to begin with. Closer debugging revealed that the cleaning step caused the detachment of these wing parts from the main body, leading to the observed artifacts. Similarly, while the humanoid example (Figure 17) only moderately overshot the target range (11,304 vertices), it illustrates how the meshing process introduced numerous holes throughout the body structure.

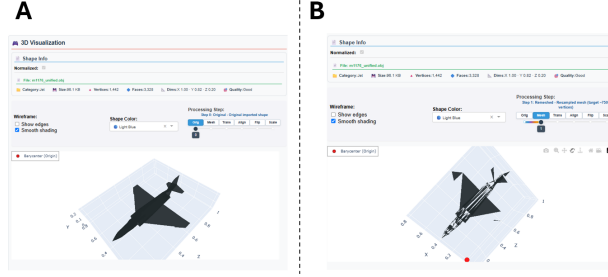


Figure 15: **Jet example: outlier meshed shape.** (A) The original object (m1176.obj), consisting of 213 vertices and 642 faces. (B) The same object after the meshing step, now consisting of 15,794 vertices and 53,248 faces.

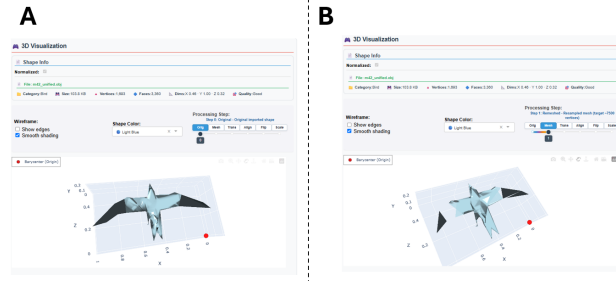


Figure 16: **Bird example: outlier meshed shape.** (A) The original object (m42.obj), consisting of 116 vertices and 265 faces. (B) The same object after the meshing step, now consisting of 1,693 vertices and 3,360 faces.

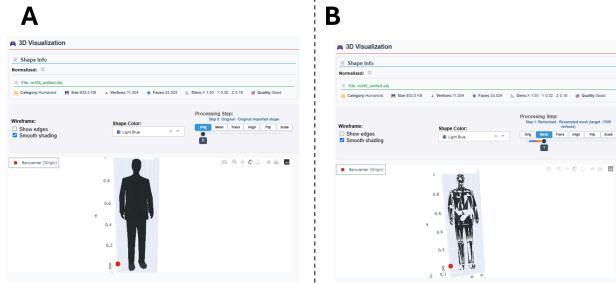


Figure 17: **Humanoid example: outlier meshed shape.** (A) The original object (m165.obj), consisting of 506 vertices and 1,748 faces. (B) The same object after the meshing step, now consisting of 11,304 vertices and 33,024 faces.

Based on these observations, we established a systematic criterion for handling outliers. As shown in Table 2, we applied a 15% tolerance threshold around the target vertex range (5,000–10,000). Shapes whose resampled vertex counts fell within this tolerance - between 4,250 and 11,500 vertices - were retained in the dataset, as minor deviations likely reflected the inherent variability of the meshing process rather than fundamental structural problems. This allowed us to keep 18 shapes that were close to the target range. Notably, of the three examples discussed above, only the humanoid (Figure 17) met this criterion with 11,304 vertices,

falling just above the target range. While it exhibited structural holes, we retained it with the expectation that subsequent hole-filling step in our pipeline would address these defects. In contrast, the jet and bird examples were excluded, as they exceeded the tolerance threshold. More broadly, the 45 excluded shapes typically exhibited the same underlying issues we identified in our detailed inspection: poor original sampling that led to holes, detached components, or excessive distortion during remeshing. Ultimately, this process yielded a cleaner, more reliable dataset of 2,438 shapes for subsequent analysis.

Table 2: **Shapes outside the target vertex range.** Among the 63 shapes identified, 18 were retained while 45 were excluded based on a 15% tolerance criterion.

extbFile	Category	Original Vertices	Resampled Vertices	Original Faces	Resampled Faces	Percent diff (%)	To Exclude
m51.obj	Bird	309	4794	598	9568	-4.12%	FALSE
m396.obj	BuildingNonResidential	445	4585	470	7520	-8.30%	FALSE
D00802.obj	Gun	300	4530	564	9024	-9.40%	FALSE
m439.obj	House	356	4346	532	8512	-13.08%	FALSE
m359.obj	HumanHead	314	4868	615	9616	-2.64%	FALSE
m361.obj	HumanHead	295	4487	557	8816	-10.26%	FALSE
m165.obj	Humanoid	506	11304	1748	33024	13.04%	FALSE
m166.obj	Humanoid	476	11239	1698	32896	12.39%	FALSE
m198.obj	Humanoid	4657	10620	19582	18680	6.20%	FALSE
m22.obj	Insect	333	4983	620	9920	-0.34%	FALSE
m1249.obj	Jet	310	4930	616	9856	-1.40%	FALSE
m1298.obj	Jet	301	4402	542	8672	-11.96%	FALSE
m633.obj	MusicalInstrument	524	4757	1728	10064	-4.86%	FALSE
m1029.obj	PlantIndoors	316	4408	548	8704	-11.84%	FALSE
D00842.obj	Spoon	291	4443	1647	8784	-11.14%	FALSE
m720.obj	Sword	318	4837	620	9664	-3.26%	FALSE
m721.obj	Sword	275	4250	530	8480	-15.00%	FALSE
m1112.obj	Tool	316	4875	621	9648	-2.50%	FALSE
m74.obj	AquaticAnimal	162	2548	336	4992	-49.04%	TRUE
m42.obj	Bird	116	1693	265	3360	-66.14%	TRUE
m487.obj	Bottle	133	1969	290	3776	-60.62%	TRUE
m471.obj	BuildingNonResidential	118	1903	248	3744	-61.94%	TRUE
m475.obj	BuildingNonResidential	112	1492	178	2816	-70.16%	TRUE
D01139.obj	Door	31	289	29	464	-94.22%	TRUE
D01143.obj	Door	48	588	72	1152	-88.24%	TRUE
m622.obj	FloorLamp	54	732	88	1408	-85.36%	TRUE
m623.obj	FloorLamp	78	967	120	1920	-80.66%	TRUE
m624.obj	FloorLamp	46	610	76	1216	-87.80%	TRUE
D00937.obj	Glasses	140	2210	280	4416	-55.80%	TRUE
m7.obj	Insect	265	4059	506	8048	-18.82%	TRUE
m1176.obj	Jet	213	15794	642	53248	57.94%	TRUE
m1213.obj	Jet	254	4012	504	8000	-19.76%	TRUE
m1270.obj	Jet	295	3883	452	7232	-22.34%	TRUE
m1294.obj	Jet	257	3131	352	5632	-37.38%	TRUE
m1295.obj	Jet	204	2274	244	3904	-54.52%	TRUE
D01120.obj	Knife	132	2102	798	4192	-57.96%	TRUE
D01160.obj	Knife	270	4169	1686	8096	-16.62%	TRUE
D01175.obj	Knife	87	1332	498	2656	-73.36%	TRUE
m998.obj	PlantIndoors	318	4125	478	7648	-17.50%	TRUE
m975.obj	PlantWildNonTree	241	3616	450	7200	-27.68%	TRUE
D00188.obj	RectangleTable	80	980	120	1920	-80.40%	TRUE
m877.obj	RectangleTable	56	686	84	1344	-86.28%	TRUE
m892.obj	RectangleTable	174	1885	288	3280	-62.30%	TRUE
m893.obj	RectangleTable	184	2458	292	4672	-50.84%	TRUE
m910.obj	RectangleTable	352	3171	1136	6512	-36.58%	TRUE
D00764.obj	Rocket	218	3218	400	6400	-35.64%	TRUE
m931.obj	RoundTable	54	714	88	1408	-85.72%	TRUE
m1448.obj	Ship	182	2683	328	5248	-46.34%	TRUE
m1670.obj	Sign	221	2147	217	3472	-57.06%	TRUE
m1671.obj	Sign	119	1145	113	1808	-77.10%	TRUE
m1672.obj	Sign	231	2652	299	4720	-46.96%	TRUE
m1673.obj	Sign	80	812	86	1376	-83.76%	TRUE
m1674.obj	Sign	50	641	78	1248	-87.18%	TRUE
m1675.obj	Sign	40	442	52	832	-91.16%	TRUE
D00674.obj	Spoon	173	2561	966	4960	-48.78%	TRUE
D00149.obj	Sword	186	2796	344	5504	-44.08%	TRUE
D00807.obj	Sword	176	2523	648	4720	-49.54%	TRUE
m689.obj	Sword	178	2824	360	5600	-43.52%	TRUE
m690.obj	Sword	180	2820	368	5600	-43.60%	TRUE
m693.obj	Sword	196	2428	259	4144	-51.44%	TRUE
m1101.obj	Tool	135	2100	262	4192	-58.00%	TRUE
m1097.obj	Tree	35	320	31	496	-93.60%	TRUE
m541.obj	Vase	312	2450	1196	5760	-51.00%	TRUE

6.6 Step 2.5: Normalizing shapes

Having reduced the variance in mesh resolution, we next address other characteristics of the dataset that require normalization. In this substep, we translate each object so that its barycenter is positioned at the origin $(0, 0, 0)$, and uniformly scale it to fit within a unit cube.

Before performing normalization, we first clean each mesh to remove potential geometric inconsistencies that could interfere with the process. This cleaning step ensures that all meshes are topologically valid and free of redundant or ill-defined elements. Specifically, we remove degenerate triangles, duplicated vertices and faces, non-manifold edges, and unreferenced vertices. By doing so, we prevent numerical instabilities during translation and scaling, and ensure that the subsequent computation of the barycenter and bounding box reflects the true geometry of the object.

Once the meshes have been cleaned, we proceed to compute their mass barycenter, which represents the center of mass under the assumption of uniform surface density. In practice, this means that each face contributes to the barycenter in proportion to its area. Rather than simply averaging all vertex coordinates, we therefore compute an area-weighted average of the centroids of all triangular faces, providing a more accurate and robust estimate of the object’s geometric center.

For a triangular mesh $M = (V, F)$ with vertices $V = \{v_1, \dots, v_n\}$, $v_i \in \mathbb{R}^3$, and faces $F = \{f_1, \dots, f_m\}$, each face $f_j = (v_{j1}, v_{j2}, v_{j3})$ contributes as follows:

$$c(f_j) = \frac{1}{3}(v_{j1} + v_{j2} + v_{j3}) \quad (1)$$

$$A(f_j) = \frac{1}{2} \|(v_{j2} - v_{j1}) \times (v_{j3} - v_{j1})\| \quad (2)$$

where $c(f_j)$ is the centroid of face f_j and $A(f_j)$ its area. The mass barycenter of the full mesh is then given by

$$b(M) = \frac{\sum_{j=1}^m A(f_j) c(f_j)}{\sum_{j=1}^m A(f_j)} \quad (3)$$

Before performing this calculation, we remove faces with an area of zero, to ensure a well-defined barycenter for all meshes in the dataset.

We have calculated the offset of the barycenter to the origin of the space before applying the translation normalization. The results can be seen in Figure 18. We observe a large variance in this offset over the dataset.

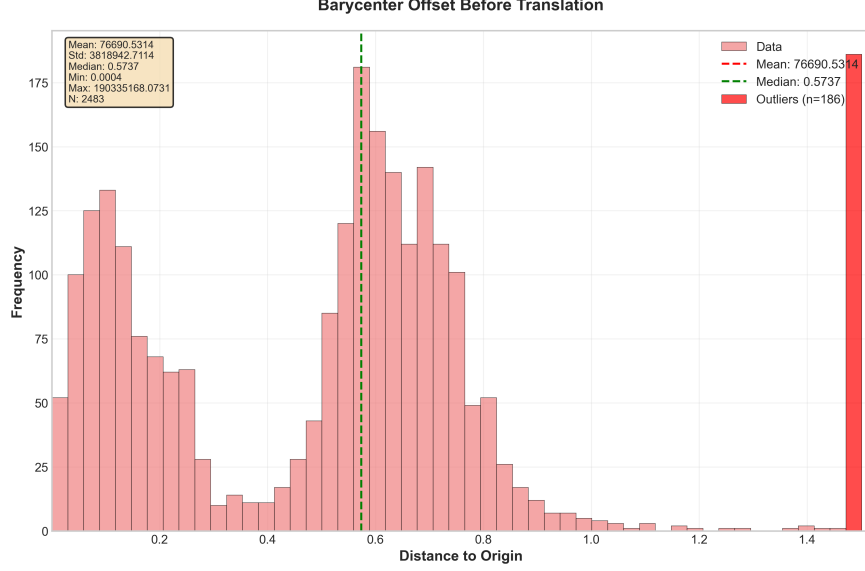


Figure 18: **Distribution of barycenter offsets before normalization.** Histogram of the Euclidean distance between each mesh barycenter and the origin. The horizontal axis shows the distance (binned), while the vertical axis shows the number of shapes. The dotted line indicates the mean offset. The red bin indicates the outliers.

Using the earlier described barycenter calculation, we translate all objects such that their barycenter is located at the origin of the space. After applying this normalization step, we again analyzed the offset of the barycenter relative to the origin. The resulting histogram can be found in Figure 19.

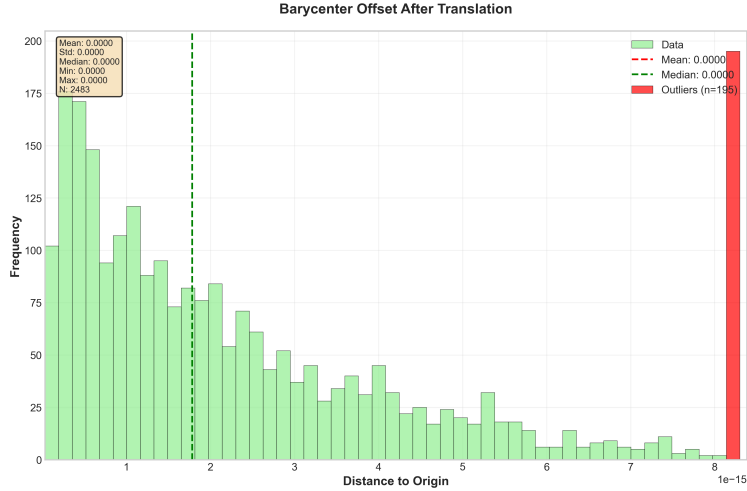


Figure 19: **Distribution of barycenter offsets after normalization.** Histogram of the Euclidean distance between each mesh barycenter and the origin. The horizontal axis shows the distance (binned), while the vertical axis shows the number of shapes. The dotted line indicates the mean offset, which is now centered around zero. (Note the small scale of the horizontal axis.)

We can see that the barycenters of all objects are now correctly positioned at the origin of the space, with

even the outliers falling well below an offset of $1e-4$.

Following the translation, we normalize the scale of each mesh. Formally, we compute the axis-aligned bounding box of a mesh $\mathbf{M}$, defined as

$$B(\mathbf{M}) = [\mathbf{v}_{\min}, \mathbf{v}_{\max}] \quad (4)$$

where

$$\mathbf{v}_{\min} = (\min_i \mathbf{v}_i^x, \min_i \mathbf{v}_i^y, \min_i \mathbf{v}_i^z), \quad \mathbf{v}_{\max} = (\max_i \mathbf{v}_i^x, \max_i \mathbf{v}_i^y, \max_i \mathbf{v}_i^z) \quad (5)$$

We then determine the bounding box side lengths as

$$\mathbf{s} = \mathbf{v}_{\max} - \mathbf{v}_{\min} \quad (6)$$

and take the largest dimension

$$s_{\max} = \max(s_x, s_y, s_z) \quad (7)$$

To enforce uniform scaling, each vertex is scaled by the factor

$$\alpha = \frac{1}{s_{\max}} \quad (8)$$

such that the longest side of the bounding box is mapped to length 1.0. This guarantees that every object is contained within a unit cube $[-0.5, 0.5]^3$.

In Figure 20 we show a histogram of the bounding box longest side length before this normalization step. Although a large number of objects already fit precisely within a unit cube bounding box, suggesting that they are already normalized to some degree, we can still observe quite a large variance in this characteristic over the dataset. After applying the scaling procedure, the distribution collapses to a single value of 1.0, as illustrated in Figure 21.

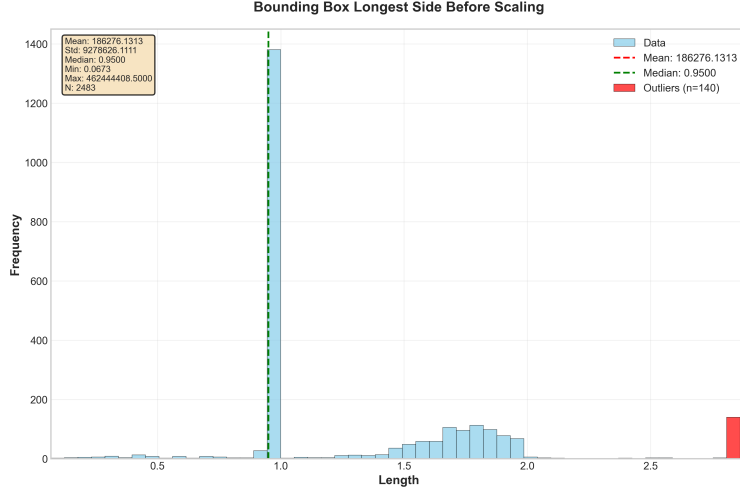


Figure 20: **Distribution of bounding box sizes before normalization.** Histogram of the longest side length of the bounding box per shape. The horizontal axis shows side length, while the vertical axis shows the number of shapes.

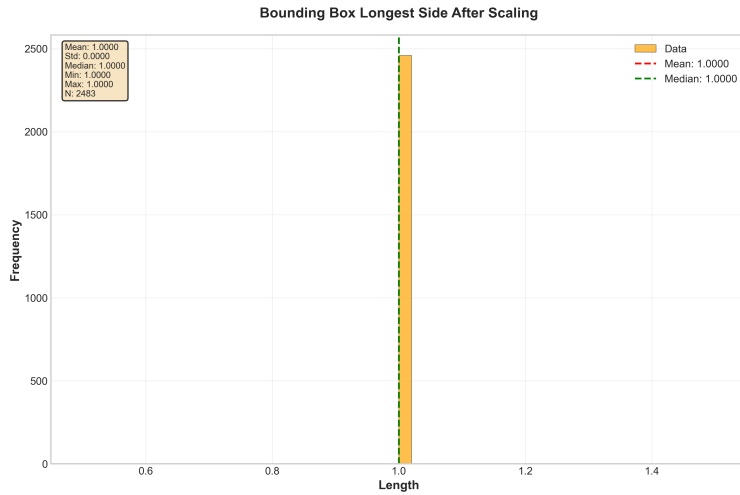


Figure 21: **Distribution of bounding box sizes after normalization.** Histogram of the longest side length of the bounding box per shape. All objects have been normalized to fit inside the unit cube, leading to uniform scaling.

Together, these two normalization steps - translating meshes to the origin and scaling them to the unit cube - ensure that all objects are centered, uniformly scaled, and thus more suitable to be compared directly. This eliminates biases due to arbitrary positioning or size differences in the raw dataset, enabling fairer and more consistent feature extraction in subsequent stages.

7 Step 3: Feature extraction

Feature extraction constitutes a critical component of content-based shape retrieval, transforming raw geometric data into compact, discriminative descriptors that capture essential shape characteristics. The

effectiveness of any retrieval system fundamentally depends on the quality and robustness of these extracted features, which must be invariant to irrelevant transformations while remaining sensitive to meaningful geometric differences.

This stage consists of two principal components: normalization and descriptor computation. Before features can be meaningfully extracted and compared, all shapes must undergo a rigorous normalization pipeline to eliminate variations in position, orientation, scale, and mesh resolution. This preprocessing ensures that extracted features reflect intrinsic shape properties rather than arbitrary differences in how the meshes were originally authored or captured.

Following normalization, we compute a comprehensive set of shape descriptors spanning both global properties and distribution-based features. Global properties include surface area, volume, and eccentricity, which characterize the overall shape structure. Distribution-based features capture geometric relationships through statistical sampling: the angle between three random points, the distance from barycenter to a random point, the area of triangles formed by three random points, the volume of tetrahedra formed by four random points, and the distance between two random points. These descriptors are designed to capture complementary aspects of shape geometry, providing a rich feature space for subsequent similarity matching.

The implementation of this stage required careful attention to numerical stability, computational efficiency, and compliance with established best practices. We leverage an enhanced mesh class for normalization operations and a dedicated feature extraction class for descriptor computation, with comprehensive validation at each step to ensure correctness.

7.1 Step 3.1: Full normalization

Normalization serves to eliminate extrinsic variations among shapes, ensuring that subsequent feature extraction focuses on intrinsic geometric properties. Our implementation follows a comprehensive five-step normalization pipeline executed in a specific order to maintain mathematical correctness: remeshing, translation, alignment (pose), flipping (orientation), and scaling.

7.1.1 Pipeline Overview

The normalization process builds upon the remeshing procedure described in Section 6, which standardizes mesh resolution across the dataset to a target range of 5,000 to 10,000 vertices. This remeshing step forms the foundation of our preprocessing pipeline, ensuring that all shapes have comparable geometric complexity before applying geometric transformations. The complete pipeline processes the shapes through the following sequence:

1. **Remeshing:** Standardize mesh resolution (as detailed in 6.3, Step 2.3. Resampling outliers)
2. **Translation:** Center the shape at the coordinate origin
3. **Alignment:** Rotate to align principal axes with coordinate axes
4. **Flipping:** Ensure consistent orientation across similar shapes
5. **Scaling:** Fit within a unit bounding box

Each transformation is applied sequentially, with the output of one step serving as input to the next. This order is critical for mathematical correctness. Centering must precede alignment because PCA requires centered data to compute meaningful principal axes. Alignment must precede flipping to establish a consistent reference frame before determining orientation. Scaling must occur last to preserve the geometric relationships established by the previous transformations.

7.1.2 Mathematical Framework

Let $M = (V, F)$ represent a triangular mesh with vertices $V = \{v_1, \dots, v_n\}$ where $v_i \in \mathbb{R}^3$ and faces F . After remeshing, the normalization pipeline applies a sequence of transformations:

$$V' = (T_4 \circ T_3 \circ T_2 \circ T_1)(V) \quad (9)$$

where T_1, T_2, T_3, T_4 represent translation, alignment, flipping, and scaling transformations, respectively, and the composition $(T_4 \circ T_3 \circ T_2 \circ T_1)(V) = T_4(T_3(T_2(T_1(V))))$ indicates that transformations are applied sequentially from right to left.

Step 1: Remeshing. As described in Subsection 6.3, [Step 2.3. Resampling outliers](#), each mesh is first evaluated against a target vertex range of 5,000 to 10,000 vertices. Meshes already within this acceptable range are identified and move on to the next step without modification, avoiding unnecessary processing that could introduce artifacts or degrade geometric quality. Only meshes falling outside this range undergo adaptive subdivision or decimation to reach the target resolution. This conditional processing ensures that all shapes are represented at comparable levels of detail while preserving the quality of already well-sampled meshes. The approach prevents high-resolution meshes from dominating feature computations and ensures low-resolution meshes have sufficient geometric information, without unnecessarily reprocessing shapes that already meet the quality criteria.

Step 2: Translation (Centering). The remeshed shape is centered at the coordinate frame origin using the mass barycenter $\mathbf{b}(M)$, computed as an area-weighted average of triangle centroids as defined in 6.6, [Step 2.5: Normalizing shapes](#):

$$\mathbf{b}(M) = \frac{\sum_{t \in T} A_t \cdot \mathbf{c}_t}{\sum_{t \in T} A_t} \quad (10)$$

where T denotes the set of triangular faces, A_t is the area of triangle t , and $\mathbf{c}_t$ is its centroid. For degenerate cases where $\sum_{t \in T} A_t < \epsilon_{\text{area}}$ (we use $\epsilon_{\text{area}} = 10^{-12}$), we fall back to the simple vertex mean. The translation transformation is:

$$v_i^{(1)} = v_i - \mathbf{b}(M) \quad (11)$$

To ensure numerical precision, we apply a two-pass recentering approach. After the initial translation, we recompute the residual barycenter $\mathbf{b}_{\text{res}} = \mathbf{b}(V^{(1)})$. If $\|\mathbf{b}_{\text{res}}\| > \epsilon_{\text{recenter}}$ (with $\epsilon_{\text{recenter}} = 10^{-9}$), we apply a correction:

$$v_i^{(1)} \leftarrow v_i^{(1)} - \mathbf{b}_{\text{res}} \quad (12)$$

This two-pass approach addresses numerical drift and degeneracy effects that can occur with large meshes or poorly conditioned geometry, ensuring that the mesh is precisely centered even in challenging cases.

Step 3: Alignment (Pose Normalization). We align the shape’s principal axes with the coordinate frame using Principal Component Analysis (PCA) [12] on the centered vertices. We employ the PCA implementation from Scikit-learn [13], which efficiently computes the covariance matrix of the vertex positions and performs eigenvalue decomposition to obtain three orthogonal eigenvectors.

Let $V = \{v_1, \dots, v_n\} \subset \mathbb{R}^3$ be the vertex positions after centering (Step 2). We compute the sample covariance matrix of the centered vertices

$$\Sigma = \frac{1}{n} \sum_{j=1}^n (v_j - \bar{v})(v_j - \bar{v})^\top, \quad (13)$$

where $\bar{v}$ is the arithmetic mean of the centered vertices (after the area-weighted centering correction in Step 2). We then obtain the spectral decomposition (eigendecomposition) of Σ

$$\Sigma \mathbf{u}_i = \lambda_i \mathbf{u}_i, \quad i \in \{1, 2, 3\}, \quad (14)$$

yielding three real eigenvalues $\{\lambda_i\}$ and corresponding orthogonal eigenvectors $\{\mathbf{u}_i\}$.

In our implementation (see `normalize_database.py`), we explicitly sort the eigenpairs by descending eigenvalue magnitude so that

$$\lambda_1 \geq \lambda_2 \geq \lambda_3 \geq 0, \quad (15)$$

and reorder the associated eigenvectors accordingly. This sorting step guarantees a consistent identification of the major, medium and minor principal axes (major $\leftrightarrow \lambda_1$, medium $\leftrightarrow \lambda_2$, minor $\leftrightarrow \lambda_3$) and is essential for downstream steps that assume this axis ordering (for example, the flipping rule and the subsequent scaling). Sorting also improves robustness when comparing axis directions across shapes.

To enforce a right-handed orthonormal basis we then form

$$\mathbf{e}_1 \leftarrow \frac{\mathbf{u}_1}{|\mathbf{u}_1|}, \quad \mathbf{e}_2 \leftarrow \frac{\mathbf{u}_2}{|\mathbf{u}_2|}, \quad (16)$$

and compute the third axis as the cross product

$$\mathbf{e}_3 := \mathbf{e}_1 \times \mathbf{e}_2, \quad (17)$$

followed by normalization

$$\mathbf{e}_3 \leftarrow \frac{\mathbf{e}_3}{|\mathbf{e}_3|}. \quad (18)$$

Recomputing $\mathbf{e}_3$ from $\mathbf{e}_1, \mathbf{e}_2$ ensures a numerically consistent right-handed frame even in cases where the raw eigendecomposition introduces small sign or ordering inconsistencies. In degenerate or nearly isotropic cases (very close eigenvalues), the code falls back gracefully: the sorting preserves a deterministic ordering and the cross-product step enforces orthogonality and handedness. Finally, each centered vertex is projected into this principal-axis frame to obtain the aligned coordinates

$$v_j^{(2)} = \begin{bmatrix} \mathbf{e}_1^\top v_j \\ \mathbf{e}_2^\top v_j \\ \mathbf{e}_3^\top v_j \end{bmatrix}, \quad j = 1, \dots, n, \quad (19)$$

i.e., by computing the dot products with the (sorted and normalized) basis vectors.

Step 4: Flipping (Orientation Consistency). Because eigenvectors are determined only up to sign, the PCA alignment above still leaves a sign ambiguity $\mathbf{e}_i \mapsto \pm \mathbf{e}_i$. To resolve this and obtain a consistent orientation across shapes we apply the moment-based sign test described below. Note that the moment test operates on the axes after we have sorted eigenpairs and enforced a right-handed basis as described in the previous step.

For each coordinate axis $i \in \{0, 1, 2\}$, we compute a signed moment:

$$f_i = \sum_{t \in T} \text{sign}(C_{t,i}) \cdot C_{t,i}^2 \quad (20)$$

where $C_{t,i}$ is the i -th coordinate of triangle t 's centroid. This quantity measures the distribution of mass along each axis, with positive values indicating that more mass lies on the positive side and negative values indicating the opposite. We then apply axis-wise flipping to ensure positive moments:

$$v_i^{(3)} = v_i^{(2)} \odot \begin{bmatrix} \text{sign}(f_0) \\ \text{sign}(f_1) \\ \text{sign}(f_2) \end{bmatrix} \quad (21)$$

where $\odot$ denotes element-wise multiplication. If $f_i = 0$ (indicating perfect symmetry), we set $\text{sign}(f_i) = 1$ to avoid unnecessary flipping. This moment test ensures that similar shapes are oriented consistently, which is essential for rotation-invariant feature extraction.

Step 5: Scaling. The final step scales the shape to fit within a unit bounding box while maintaining aspect ratios. We compute the extent of the aligned and flipped mesh along each axis:

$$d_i = \max_j v_{j,i}^{(3)} - \min_j v_{j,i}^{(3)}, \quad i \in \{0, 1, 2\} \quad (22)$$

and identify the maximum extent:

$$d_{\max} = \max(d_0, d_1, d_2) \quad (23)$$

Each vertex is then uniformly scaled by the reciprocal of this maximum extent:

$$v_i^{(4)} = \frac{v_i^{(3)}}{d_{\max}} \quad (24)$$

This uniform scaling ensures that the longest dimension of the bounding box equals 1.0 while preserving the shape’s aspect ratios. All normalized shapes thus fit within a unit cube centered at the origin.

Following the scaling step, we apply a final safety check by recomputing the barycenter $\mathbf{b}_{\text{final}} = \mathbf{b}(V^{(4)})$. If $\|\mathbf{b}_{\text{final}}\| > \epsilon_{\text{recenter}}$, we apply a final centering correction:

$$v_i^{(4)} \leftarrow v_i^{(4)} - \mathbf{b}_{\text{final}} \quad (25)$$

This post-scaling recenter pass ensures that the normalized mesh remains precisely centered at the origin, which is critical for rotation-invariant feature extraction and fair similarity comparisons.

7.1.3 Implementation and Validation

Our implementation processes the entire dataset through a unified preprocessing pipeline that integrates all five normalization steps. The pipeline includes several design features to ensure robustness and facilitate debugging:

- **Smart skip logic:** Already-processed shapes are detected and skipped to enable efficient re-runs and iterative refinement of the pipeline
- **Step-by-step output:** For validation purposes, we save intermediate OBJ files after each normalization step:
 - `00_original.obj`: Original mesh from the dataset
 - `01_remeshed.obj`: After adaptive remeshing (only present if remeshing was applied)
 - `02_translated.obj`: After barycenter translation
 - `03_aligned.obj`: After PCA alignment
 - `04_flipped.obj`: After moment-based flipping
 - `05_scaled.obj`: Final normalized mesh
- **Comprehensive validation:** Each processed shape undergoes extensive validation checks:
 - Centering error analysis (distance from barycenter to origin)
 - PCA eigenvalue ordering verification (ensuring $\lambda_1 \geq \lambda_2 \geq \lambda_3$)
 - Moment test validation (confirming positive signed moments)
 - Bounding box verification (confirming maximum extent equals 1.0)

– Cross-step consistency checks (ensuring each transformation preserves mesh topology)

- **Enhanced numerical robustness:** The implementation uses area-weighted barycenter computation with fallback mechanisms for degenerate cases, two-pass recentering to handle numerical drift, and double-precision floating-point arithmetic throughout

The preprocessing pipeline ran to completion for all 2,483 shapes; while no processing terminated with a fatal error, a small subset (63) were flagged as mesh-quality outliers and have been recorded for manual review and will be discussed in sub-section 6.5. Validation metrics confirm that our implementation achieves high mathematical precision:

- Mean centering error: 1.04×10^{-14} (effectively zero within floating-point precision)
- Maximum centering error: 9.22×10^{-13} (well within numerical tolerance)
- Mean scaling error: 3.18×10^{-17} (negligible)
- Maximum scaling error: 2.22×10^{-16} (within machine epsilon)
- 100% compliance with eigenvalue ordering ($\lambda_1 \geq \lambda_2 \geq \lambda_3$)
- 100% correct moment sign after flipping (all axes show positive moments)

These results demonstrate that our implementation achieves the mathematical precision required for robust feature extraction. The normalized shapes are stored in the **UnifiedPreprocessed** directory, organized by category, with accompanying metadata files containing processing statistics and validation results for each shape.

Figure 22 illustrates the average proportion of variance explained by each principal component across all normalized shapes. The dominance of PC1 (73.2%) and the much smaller contributions of PC2 (20.0%) and PC3 (6.8%) confirm that the normalization pipeline consistently aligns the major axis of variation and reduces residual variance in minor directions. This validates that the preprocessing steps - especially PCA based alignment - effectively standardize shape orientation and scale, ensuring robust and interpretable feature extraction for downstream analysis.

Average Variance Explained by Each Principal Component (N = 2483)

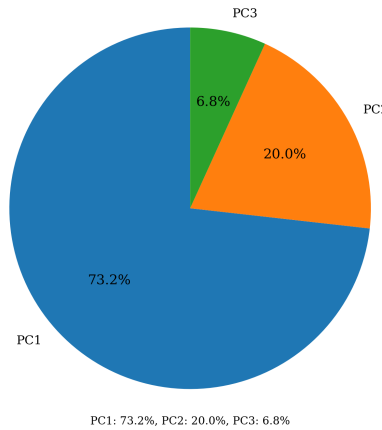


Figure 22: Average variance explained by each principal component across all validated shapes ($N = 2483$). PC1 accounts for 73.2%, PC2 for 20.0%, and PC3 for 6.8% of the total variance.

Interpretation and evidence for numerical precision for the centering step. The plot in Figure 23 was produced by extracting every shape’s final centering error from the validation metadata and plotting the

empirical cumulative distribution (CDF) of those errors on a logarithmic x -axis. The red dashed line indicates our target centering tolerance (10^{-10}), while the orange dashed line indicates a conservative numerical-noise region near machine precision (an operational marker chosen here at 10^{-13} to show where floating-point rounding and cancellation may begin to dominate). The CDF rises to 100% well before the target line, which shows that essentially all shapes have centering errors below the target tolerance. Because the bulk of the distribution lies many orders of magnitude above the machine-noise marker but below the target, the result demonstrates that centering is numerically accurate and robust (the orange marker simply highlights the scale at which numerical rounding would become relevant).

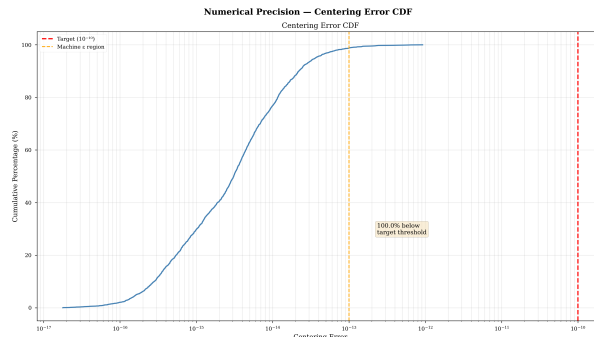


Figure 23: Centering error CDF. Empirical cumulative distribution (percentage) of final centering errors (x -axis, log scale). The red dashed line marks the target threshold (10^{-10}); the orange dashed line marks a conservative numerical-noise region associated with machine precision effects.

Interpretation and evidence for successful scaling. The multi-panel scatter figure (Figure 24) was generated by reading each shape’s metadata (which was saved during the normalization process) and plotting the three final bounding-box side lengths (X , Y and Z) for every shape, grouped by category; within each small subplot every marker corresponds to one shape and the unit-cube target corresponds to a maximum side length of 1.0. The figure shows that, for all categories, the per-shape maxima of $\{X, Y, Z\}$ are at or below 1.0, i.e., the colored points lie within the 0–1 range and no category exhibits a systematic pattern of values exceeding the unit scale. This visual pattern provides direct evidence that the scaling step produced consistent unit-cube normalization across the dataset.

33

7.1.4 Integration with Viewer Application

The normalized shapes are seamlessly integrated into our interactive viewer application via a dedicated caching system, enabling efficient access to pre-computed normalized vertices and eliminating redundant computation. In the shape information panel, a "Normalized" checkbox indicates whether the currently viewed shape has undergone normalization. Users interactively explore the normalization pipeline using the processing step slider, which allows inspection of each intermediate step (see Figures 25 and 26). This integration enables direct visual validation of the normalization process and supports comparative analysis of shape features before and after normalization.

The two examples in Figures 25 and 26 illustrate the impact of each normalization step on different shapes. For the plant mesh, the initial cleaning and meshing steps remove most of the fine leaf structures, resulting in a simplified mesh that primarily represents the main stem and pot. Subsequent steps - translation, alignment, flipping, and scaling - further standardize the orientation and size, but the most dramatic change occurs during cleaning and meshing. In contrast, the pig mesh starts with 3,666 vertices, already close to the target range, so the remeshing step makes only minor adjustments. The following normalization steps (translation to the origin, PCA alignment, flipping for consistent orientation, and scaling to a unit bounding box) are applied similarly, but the overall shape and details of the pig are largely preserved throughout the pipeline. These examples demonstrate how the normalization process adapts to the characteristics of each object, either simplifying geometry when necessary or maintaining detail when possible, while ensuring consistent orientation and scale across the dataset.

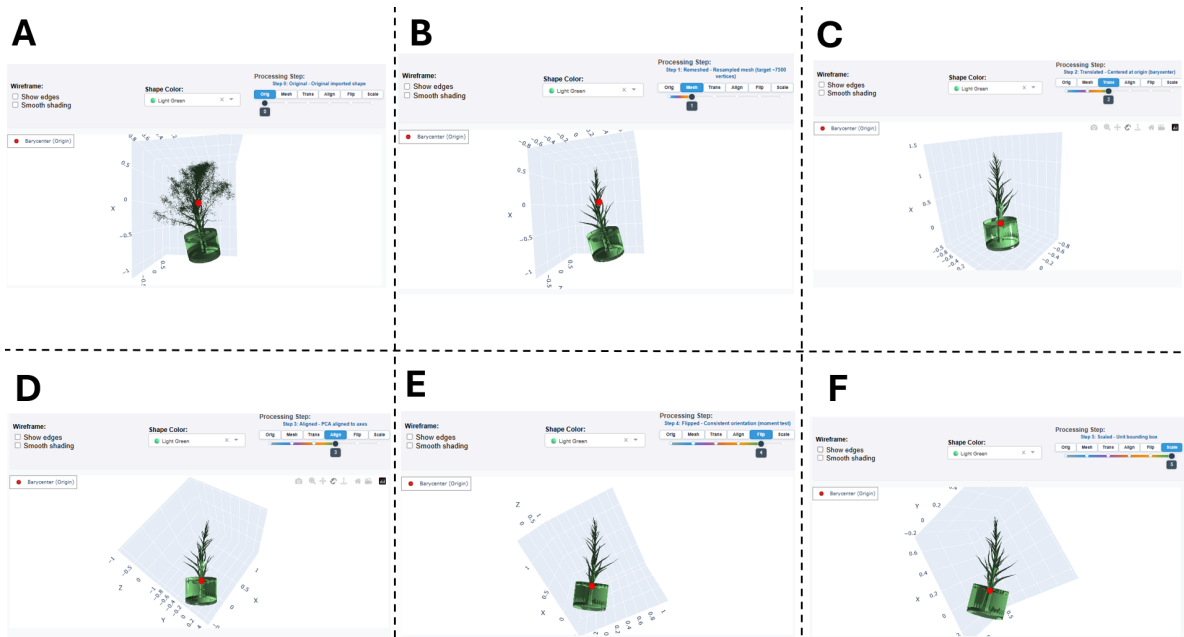


Figure 25: **Normalization steps for a plant mesh (D00080.obj).** Each panel shows the mesh at a different stage of the normalization pipeline as visualized in the viewer application. (A) Original shape. (B) Remeshed to target vertex count. (C) Translated the barycenter to origin. (D) PCA aligned to axes. (E) Flipped for consistent orientation. (F) Scaled to unit bounding box.

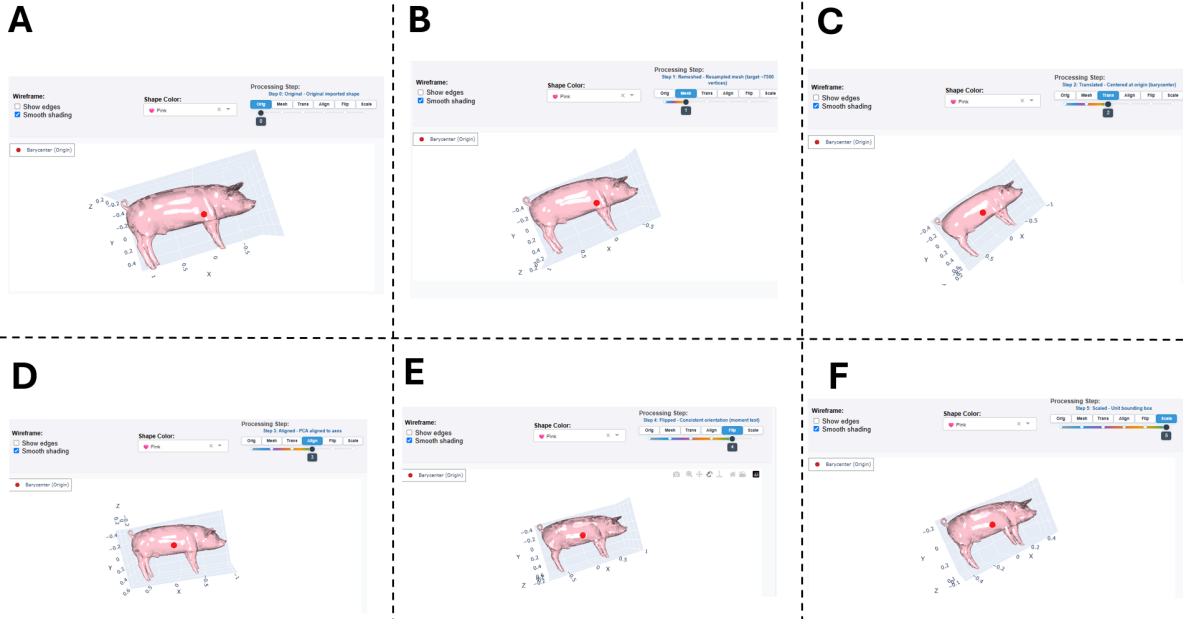


Figure 26: **Normalization steps for a pig mesh (D00218.obj).** Each panel shows the mesh at a different stage of the normalization pipeline as visualized in the viewer application. (A) Original shape. (B) Remeshed to target vertex count. (C) Translated the barycenter to origin. (D) PCA aligned to axes. (E) Flipped for consistent orientation. (F) Scaled to unit bounding box.

To further enhance usability, the viewer incorporates intelligent fallback logic for missing files. Shapes already within the acceptable vertex range skip the remeshing step and do not have a `01_remeshed.obj` file. In these cases, the slider automatically displays the `00_original.obj` file when the remeshing step is selected, ensuring uninterrupted visual inspection of the pipeline. This adaptive behavior provides transparency into the preprocessing workflow and helps users understand which transformations were necessary for each shape, supporting both quality control and informative purposes.

7.2 Step 3.2: 3D shape descriptors

The purpose of this step is to transform normalized 3D meshes into discriminative descriptors that capture essential geometric characteristics. Each descriptor is designed to be invariant to translation, rotation, and scale, while still reflecting intrinsic shape differences. The following subsections present the mathematical foundations of the descriptors we implemented.

7.2.1 Mesh preparation for feature extraction

Before computing descriptors, it is crucial to ensure that the normals of the faces are consistent and free of geometric defects. In particular, incomplete meshes with holes or inconsistent face windings can produce invalid volumes or surface areas. To address these issues, we apply a preprocessing routine consisting of two main steps: (1) filling holes and (2) orienting faces consistently.

7.2.1.1 Filling holes.

We want to fill holes because properties like volume and surface area are calculated using the faces. If some faces are missing, you do not want these descriptors to be incorrect. Boundary edges of a mesh are identified by counting the number of times each edge occurs in the face list. Edges that appear exactly once belong to the boundary of a hole. By traversing these edges, we group them into closed loops that represent the

perimeters of the holes. For each hole loop, we compute its barycenter,

$$c_{\text{hole}} = \frac{1}{|E|} \sum_{(i,j) \in E} \frac{v_i + v_j}{2}, \quad (26)$$

where E is the set of edges in the loop. This barycenter is added as a new vertex, and each boundary edge is connected to it to form a new triangular face. The process closes the hole while maintaining manifold connectivity. Although this procedure may simplify fine geometry, it is mostly good enough for the descriptors (volume, surface area) we want to compute.

7.2.1.2 Filling holes (limitations).

The barycentric triangulation approach for closing boundary loops is a pragmatic heuristic rather than a guaranteed, topology-preserving repair. Several failure modes can occur in practice:

- **Non-manifold edges:** edges incident to three or more faces break the assumption that each interior edge is shared by exactly two faces; loop detection and subsequent triangulation will fail and holes using these edges will not be repaired.
- **Dangling/isolated edges:** multiple single (boundary) edges meeting at a vertex may not form a simple cycle, preventing a well-defined hole loop, or creating holes in a weird way. for example: two holes connecting at a single vertex can be repaired with a single 'plaster'.
- **Open or branching boundaries:** when boundary tracing encounters forks or dead-ends, no closed loop exists and triangulation will not complete.
- **Severe non-planarity:** highly warped boundary rims can yield skinny or inverted triangles when all edges are connected to a single barycenter in $\mathbb{R}^3$.

In our implementation, we treat loop construction as best-effort: if a consistent cycle cannot be recovered, the routine aborts for that component and leaves the boundary unchanged, logging a warning. After any successful fill, we run a cleanup pass that removes duplicate/degenerate faces and drops unreferenced vertices to mitigate newly introduced artifacts. Consequently, hole filling *does not guarantee* watertightness; rather, it increases the likelihood of obtaining a closed, consistently oriented surface suitable for downstream feature extraction while acknowledging the above limitations.

7.2.1.3 Consistent face orientation.

Meshes may contain inconsistently oriented faces, meaning that adjacent triangles along a shared edge traverse that edge in the same direction rather than opposite directions. This creates local inconsistencies in surface normal orientation. This is a problem for several descriptors such as the volume. To correct this, we perform a breadth-first traversal over faces: starting from an arbitrary seed face, all neighbors are visited through their shared edges. Whenever two faces share an edge with the same orientation, one is flipped by swapping two of its vertex indices:

$$(i, j, k) \mapsto (i, k, j). \quad (27)$$

This ensures that the mesh forms a consistently oriented surface, suitable for all calculations of the descriptors.

7.2.1.4 Combined preparation.

The overall preparation pipeline can thus be summarized as:

1. Detect boundary edges and close holes by barycentric triangulation.
2. Remove degenerate and duplicate faces, and drop unused vertices.

3. Traverse the mesh to enforce consistent face winding.

This process produces meshes, which are then used as a more reliable input for feature extraction. Similar to the normalization steps (Subsection 7.1.3), we cached the output of this process as an OBJ file with the suffix `_06_fill_holes_and_orientation.obj`, allowing users to interactively visualize the effects of this step using the same slider controller in the UI.

7.2.2 Elementary descriptors

Before introducing the descriptors, it is important to recall that all meshes in the dataset are first *unified*. As described in section 7.1, this includes alignment, scaling, and centering. In a second step, the meshes are prepared by filling holes and equalizing normals (see section 7.2.1). As a result, every mesh is consistently placed inside the cube $[-0.5, 0.5]^3$, ensuring that all descriptors are directly comparable across objects, independent of their original position, orientation, or scale.

Feature Standardization - Each numeric descriptor was standardized using the z-score transform

$$z = \frac{x - \mu}{\sigma}, \quad (28)$$

where μ and σ are the column mean and population standard deviation, respectively. For each feature listed below, we included a combined figure, (Figures 27, 28, 30, 32, 34, 36, and 38), which includes the original distribution (left column) and the z-score standardized distribution (right column), with histograms (top row), boxplots (middle row), and a compact table of summary statistics (bottom row). After successful standardization the transformed data have a mean near 0 and a standard deviation near 1; unusually large values of $\max |z|$ or low percentages within $|z| \leq 1, 2, 3$ indicate skewness or outliers that merit further inspection.

7.2.2.1 Volume

Several descriptors rely on a robust estimate of the enclosed volume. Following our implementation, we compute the signed volume per connected face component and then sum their absolute values:

$$V(M) = \sum_{c \in \mathcal{C}(M)} \left| \frac{1}{6} \sum_{(i,j,k) \in F_c} v_i \cdot ((v_j \times v_k)) \right|, \quad (29)$$

where $\mathcal{C}(M)$ denotes the set of connected face components and F_c the faces in component c . This per-component absolute value matches the BFS accumulation in our code and is robust to local orientation inconsistencies. The resulting $V(M)$ is then reused by compactness, convexity, and (volume-based) rectangularity.

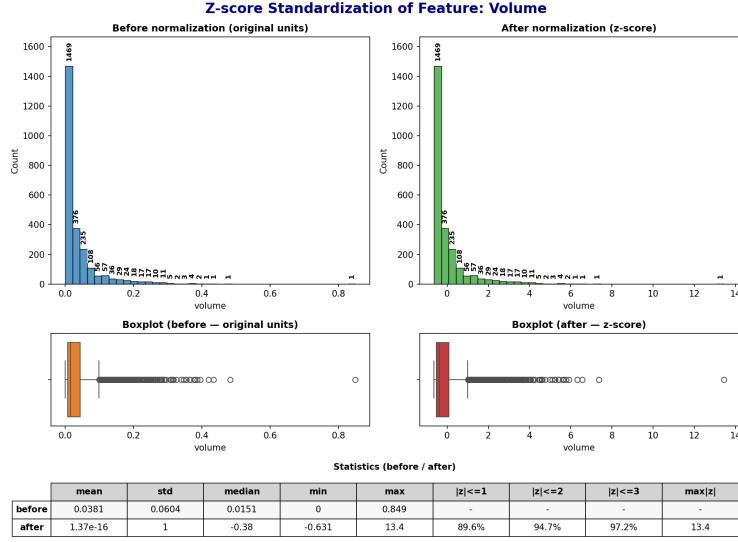


Figure 27: **Z-score standardization of Volume**. Left: raw distribution in original units (histogram and boxplot); right: z-score standardized distribution. The bottom table shows summary statistics before and after standardization (mean, std, median, min, max, and percentages within $|z| \leq 1, 2, 3$).

7.2.2.2 Surface area

The surface area describes the total extent of a mesh surface by summing the contributions of all triangular faces. It is defined as

$$A(M) = \frac{1}{2} \sum_{(i,j,k) \in F} \|(v_j - v_i) \times (v_k - v_i)\|. \quad (30)$$

Intuitively, this value captures how much space the surface covers; shapes with large, detailed, or highly folded surfaces will have larger surface areas compared to simpler ones.

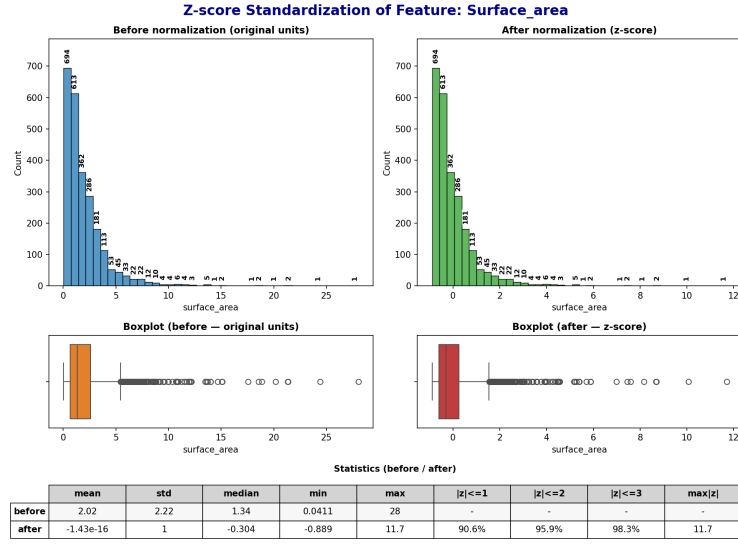


Figure 28: **Z-score standardization of Surface area.** Left: raw distribution in original units (histogram and boxplot); right: z-score standardized distribution. The bottom table shows summary statistics before and after standardization (mean, std, median, min, max, and percentages within $|z| \leq 1, 2, 3$).

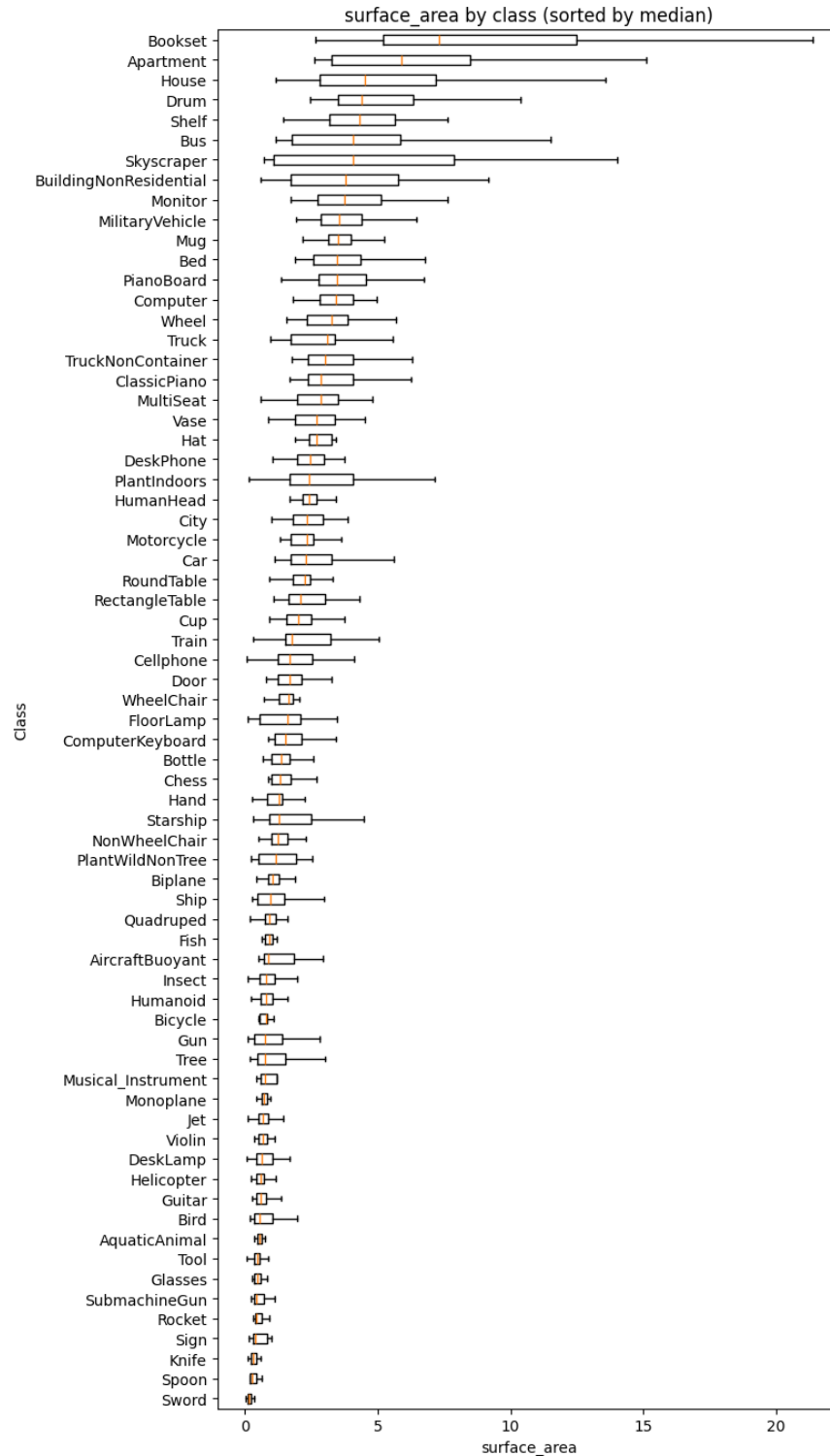


Figure 29: **Boxplot of *Surface area* values by class (sorted by median).** Each horizontal box shows the distribution of descriptor values for a specific class, with classes ordered by their median. This allows for a clear comparison of central tendencies and spread across categories.

7.2.2.3 Compactness

Compactness compares the ratio of surface area to volume against that of a perfect sphere. The formula is

$$\mathcal{C}(M) = \frac{A(M)^3}{36\pi V(M)^2}. \quad (31)$$

A value close to 1 indicates a sphere, the most compact shape, while larger values highlight shapes that are elongated, spiky, or irregular.

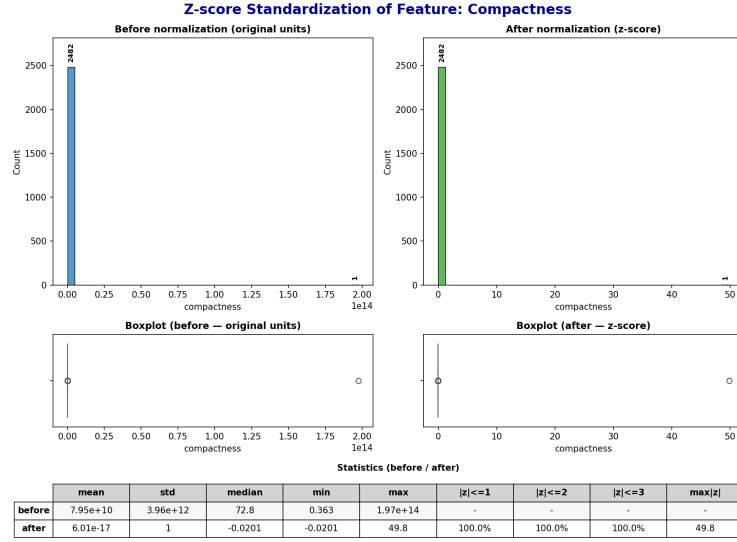


Figure 30: **Z-score standardization of Compactness.** Left: raw distribution in original units (histogram and boxplot); right: z-score standardized distribution. The bottom table shows summary statistics before and after standardization (mean, std, median, min, max, and percentages within $|z| \leq 1, 2, 3$).

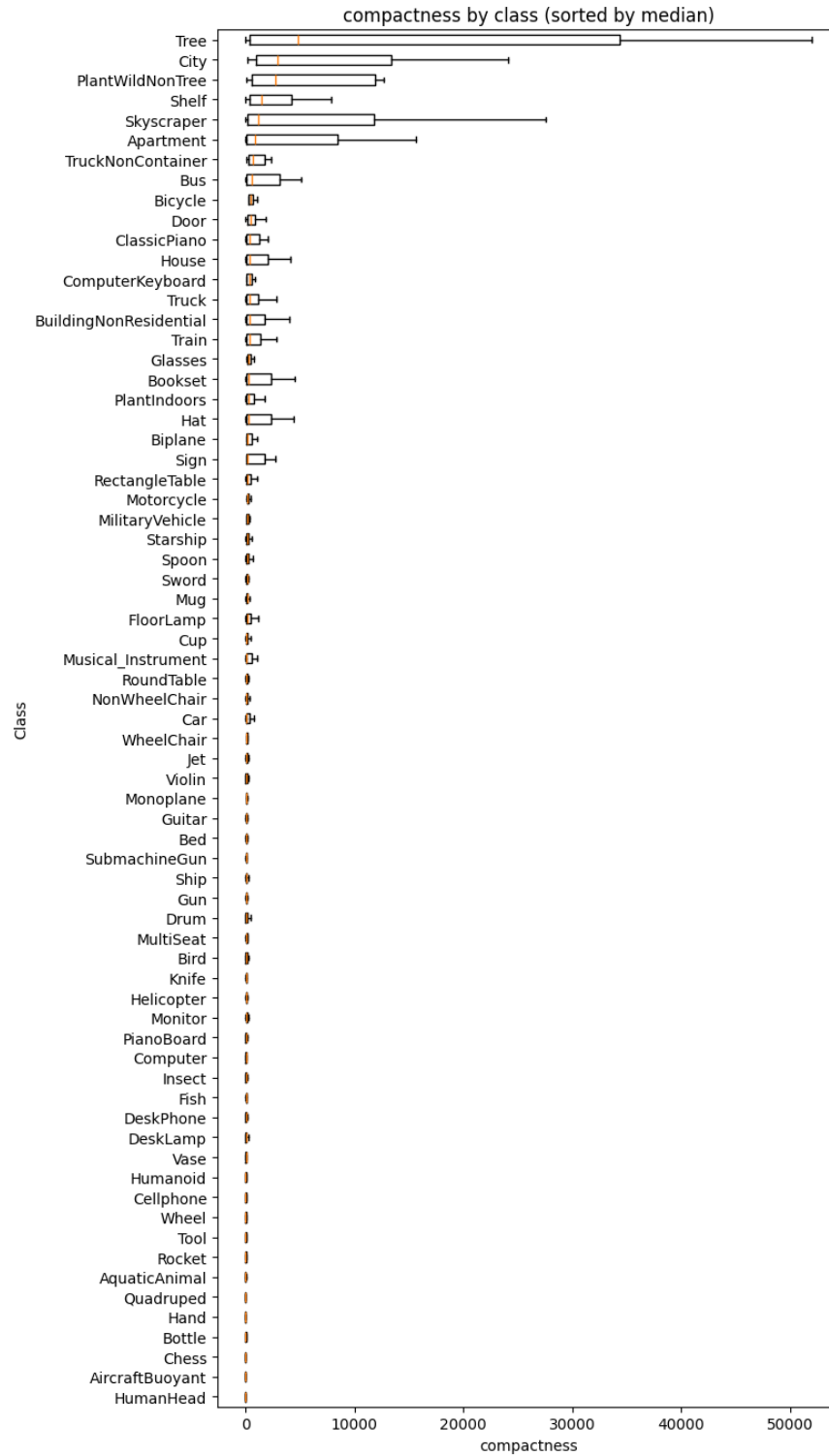


Figure 31: **Boxplot of *Compactness* values by class (sorted by median)**. Each horizontal box shows the distribution of descriptor values for a specific class, with classes ordered by their median. This highlights differences in how compact or elongated different object classes are.

7.2.2.4 Rectangularity

Rectangularity measures how well the shape *fills* its oriented bounding box (OBB) in terms of volume. If the OBB has side lengths (w, h, d) , its volume is $V_{\text{OBB}} = w h d$, and we define

$$\mathcal{R}(M) = \frac{V(M)}{V_{\text{OBB}}(M)} \in (0, 1]. \quad (32)$$

High values indicate that the object volumetrically occupies most of its OBB (box-like, chunky shapes), while low values signal thin or highly concave objects. Note that a *thin* door for example still has a high value as the OBB box is also small.

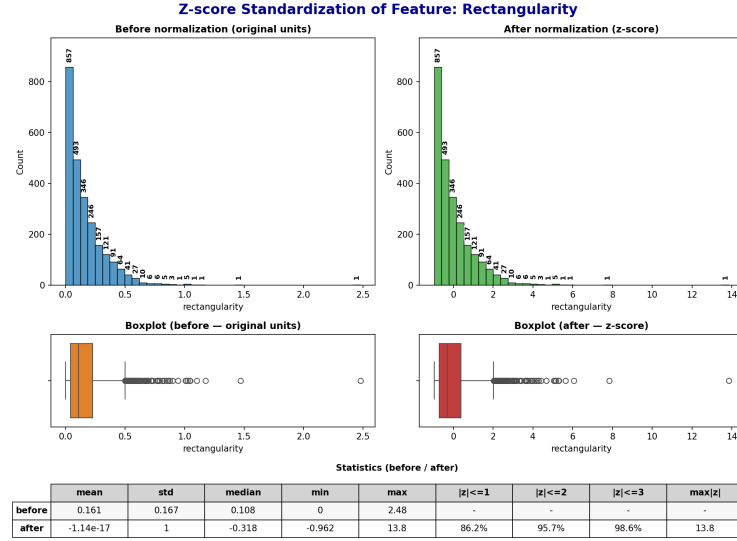


Figure 32: **Z-score standardization of Rectangularity.** Left: raw distribution in original units (histogram and boxplot); right: z-score standardized distribution. The bottom table shows summary statistics before and after standardization (mean, std, median, min, max, and percentages within $|z| \leq 1, 2, 3$).

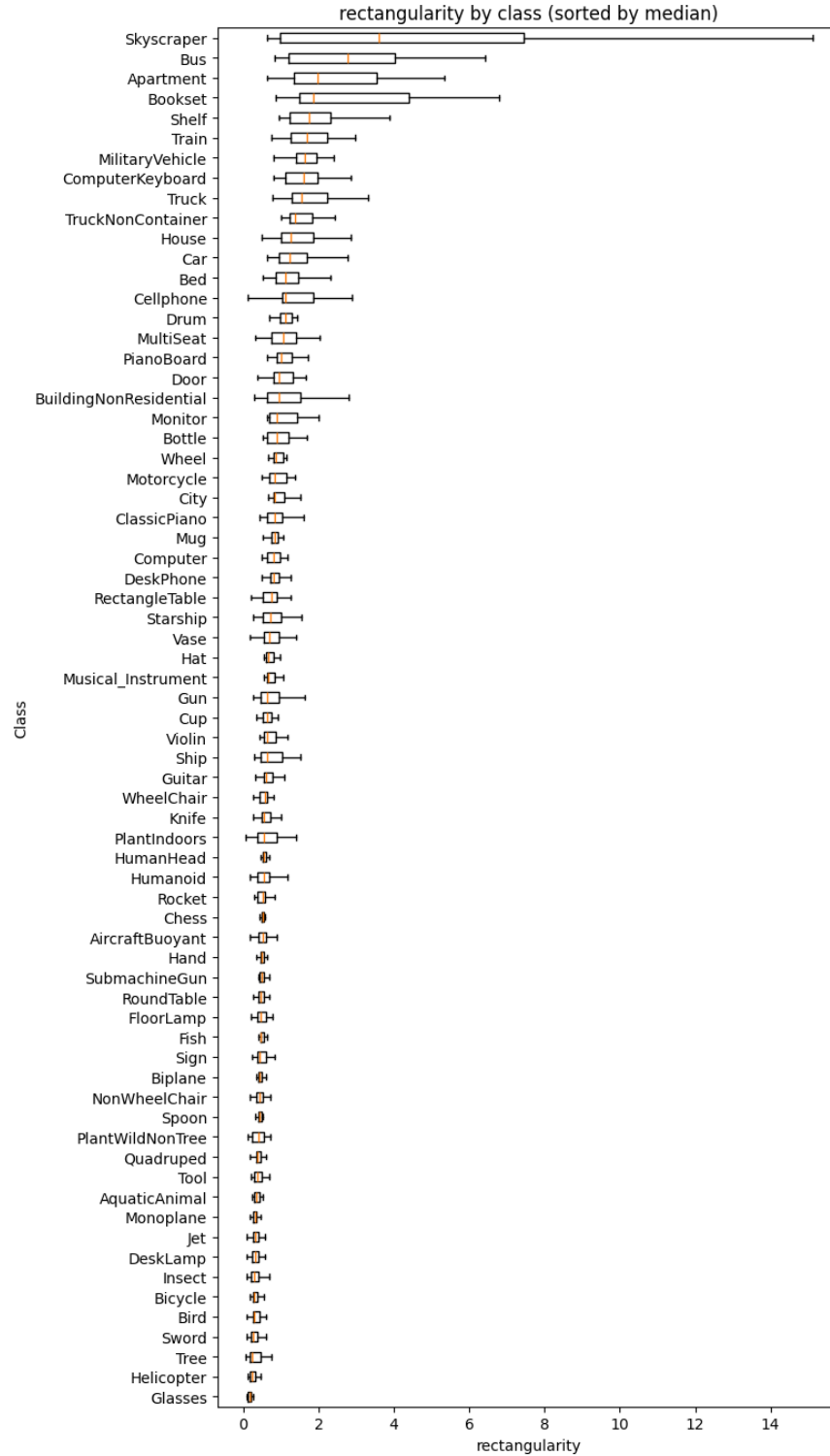


Figure 33: **Boxplot of *Rectangularity* values by class (sorted by median)**. Each box illustrates how efficiently shapes occupy their oriented bounding box volume. Classes are ordered by median rectangularity, enabling comparison of box-like versus irregular geometries.

7.2.2.5 Diameter

The diameter is the maximum Euclidean distance between any two vertices of the mesh:

$$D(M) = \max_{p,q \in V} \|p - q\|_2. \quad (33)$$

This captures the longest measurable extent of the object and helps distinguish elongated objects from more compact ones. As the objects are scaled the minimum value is 1. Higher values indicate less compact objects.

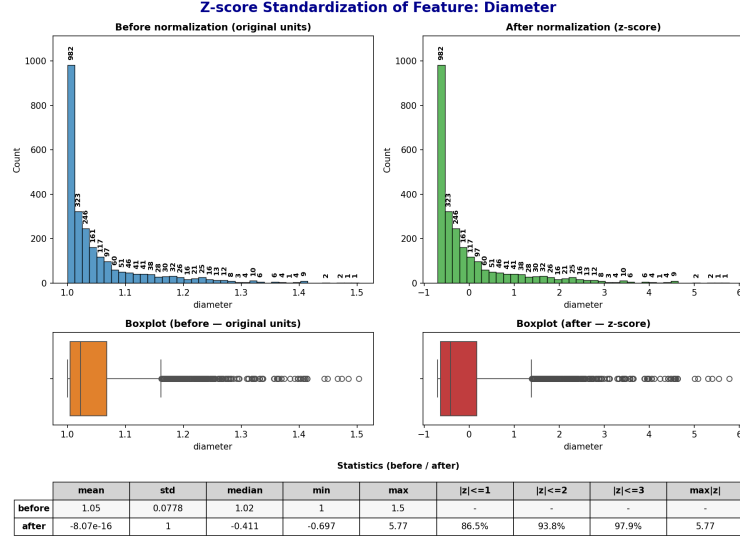


Figure 34: **Z-score standardization of Diameter.** Left: raw distribution in original units (histogram and boxplot); right: z-score standardized distribution. The bottom table shows summary statistics before and after standardization (mean, std, median, min, max, and percentages within $|z| \leq 1, 2, 3$).

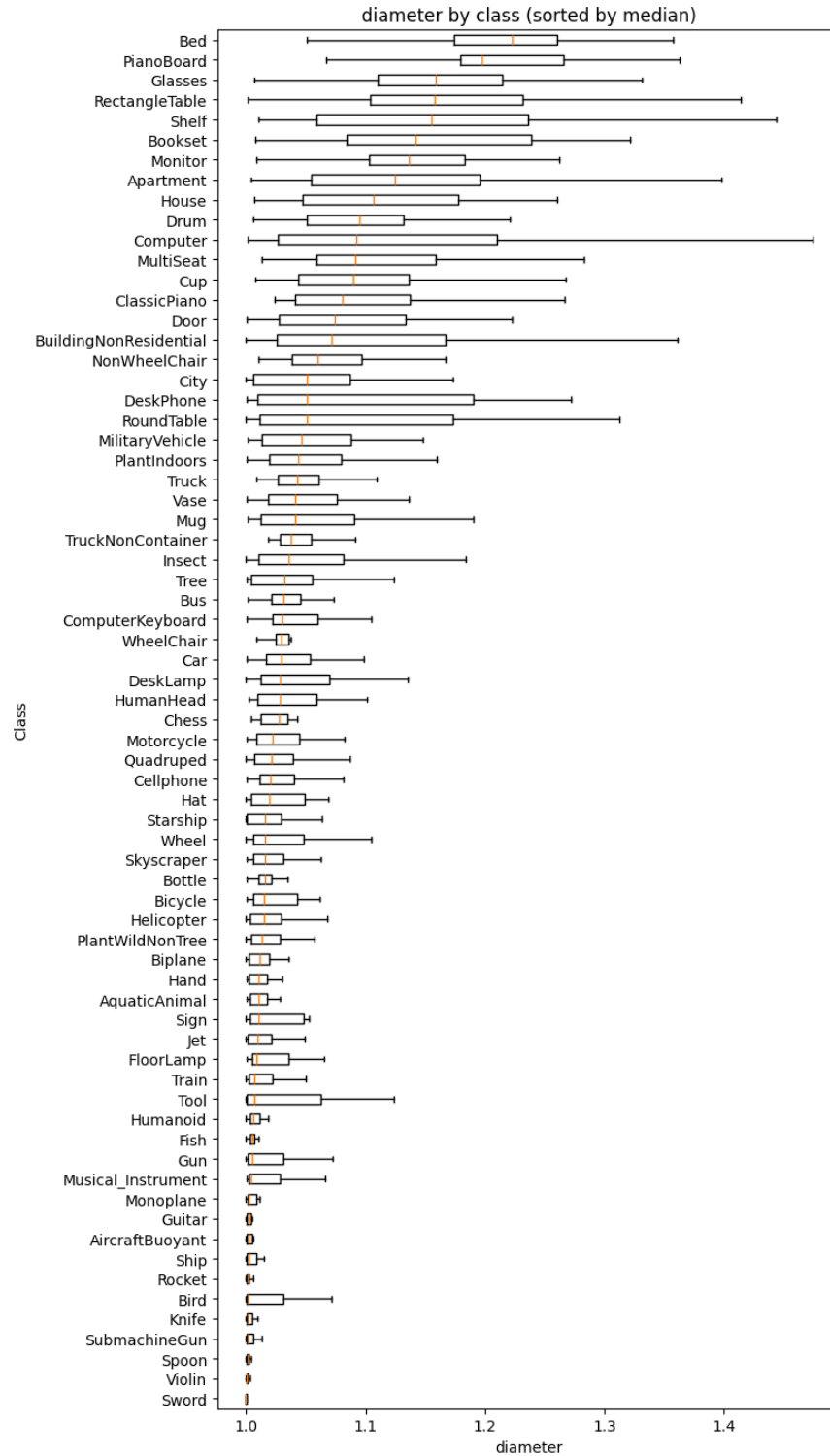


Figure 35: **Boxplot of *Diameter* values by class (sorted by median)**. The diameter captures the maximum span of each shape. This plot shows which classes tend to have longer or more compact overall extents.

7.2.2.6 Convexity

Convexity quantifies the degree to which the shape fills its convex hull. With M^{hull} denoting the convex hull,

$$\mathcal{CX}(M) = \frac{V(M)}{V(M^{\text{hull}})}. \quad (34)$$

Values close to 1 indicate convex shapes, while smaller values indicate strong concavities or internal cavities.

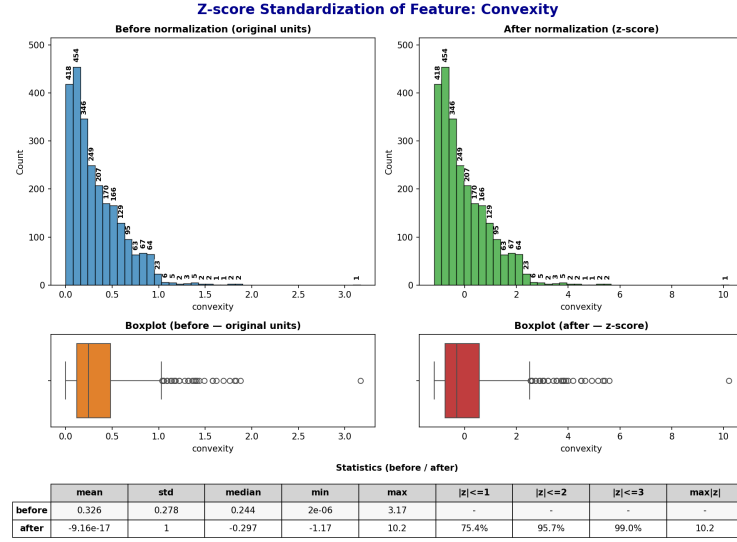


Figure 36: **Z-score standardization of Convexity.** Left: raw distribution in original units (histogram and boxplot); right: z-score standardized distribution. The bottom table shows summary statistics before and after standardization (mean, std, median, min, max, and percentages within $|z| \leq 1, 2, 3$).

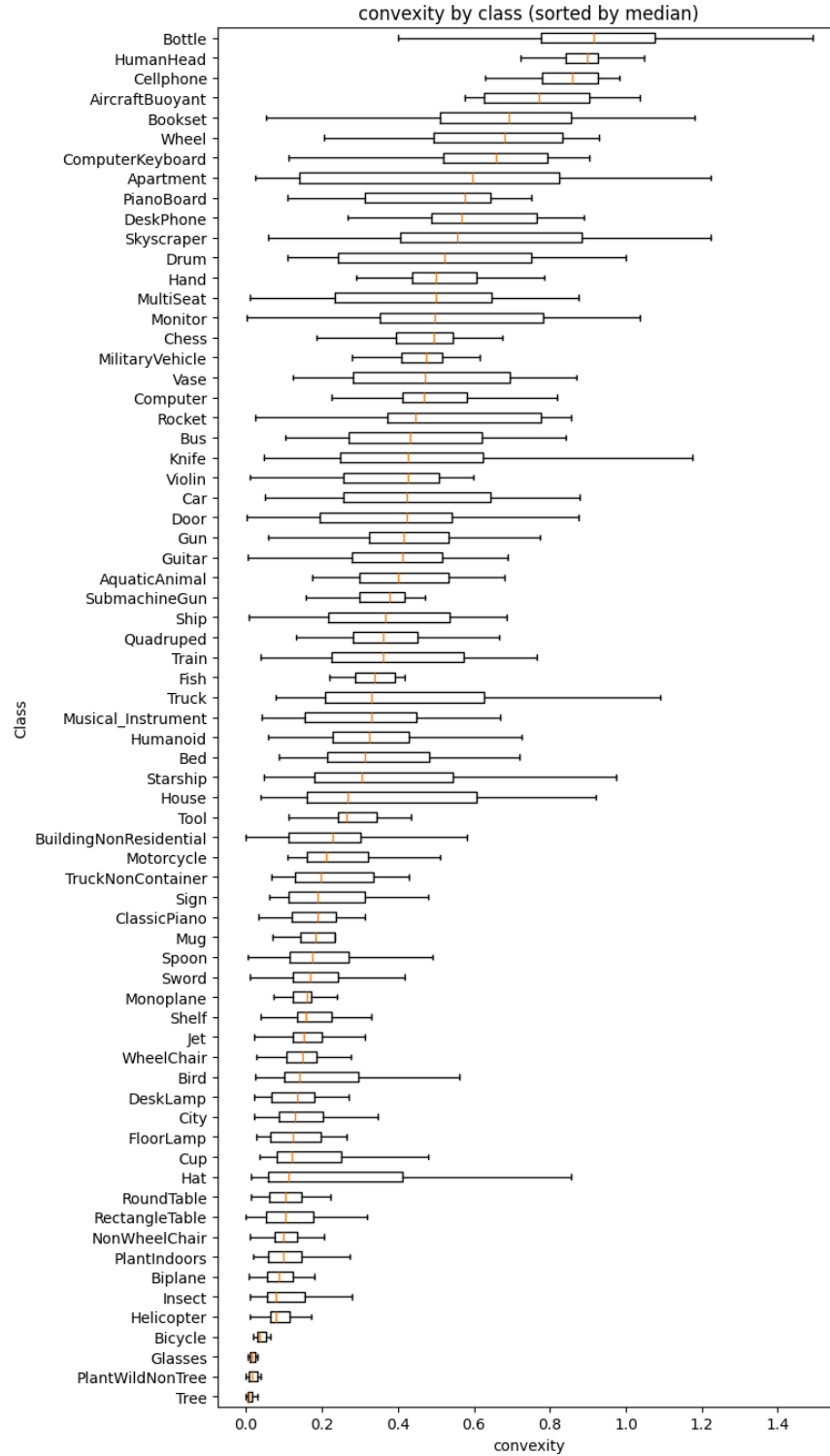


Figure 37: **Boxplot of *Convexity* values by class (sorted by median)**. Convexity measures how much of the convex hull volume is filled by the mesh. The ordering by median highlights which classes are closer to convex and which show deep concavities.

7.2.2.7 Eccentricity

Eccentricity measures how stretched a shape is along its principal axes. With eigenvalues $\lambda_1 \geq \lambda_2 \geq \lambda_3$ of the vertex covariance matrix,

$$\varepsilon(M) = \frac{\lambda_1}{\lambda_3}. \quad (35)$$

A sphere yields $\varepsilon \approx 1$, while thin or elongated objects have much higher values, capturing their anisotropy.

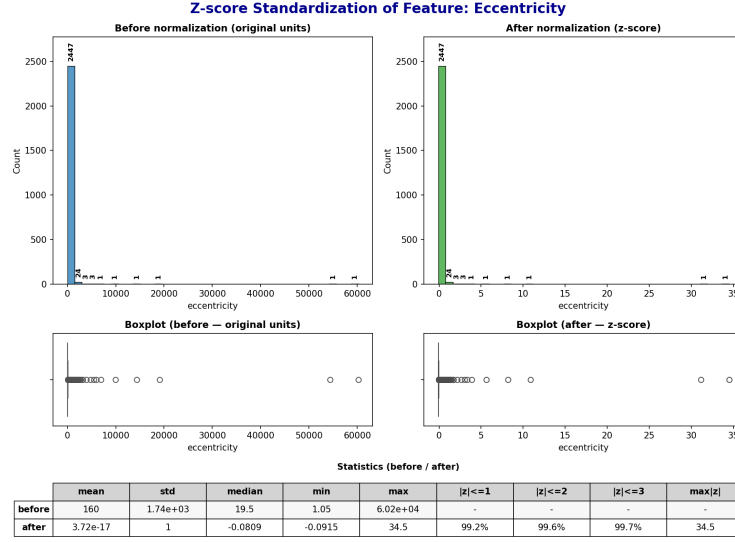


Figure 38: **Z-score standardization of Eccentricity.** Left: raw distribution in original units (histogram and boxplot); right: z-score standardized distribution. The bottom table shows summary statistics before and after standardization (mean, std, median, min, max, and percentages within $|z| \leq 1, 2, 3$).

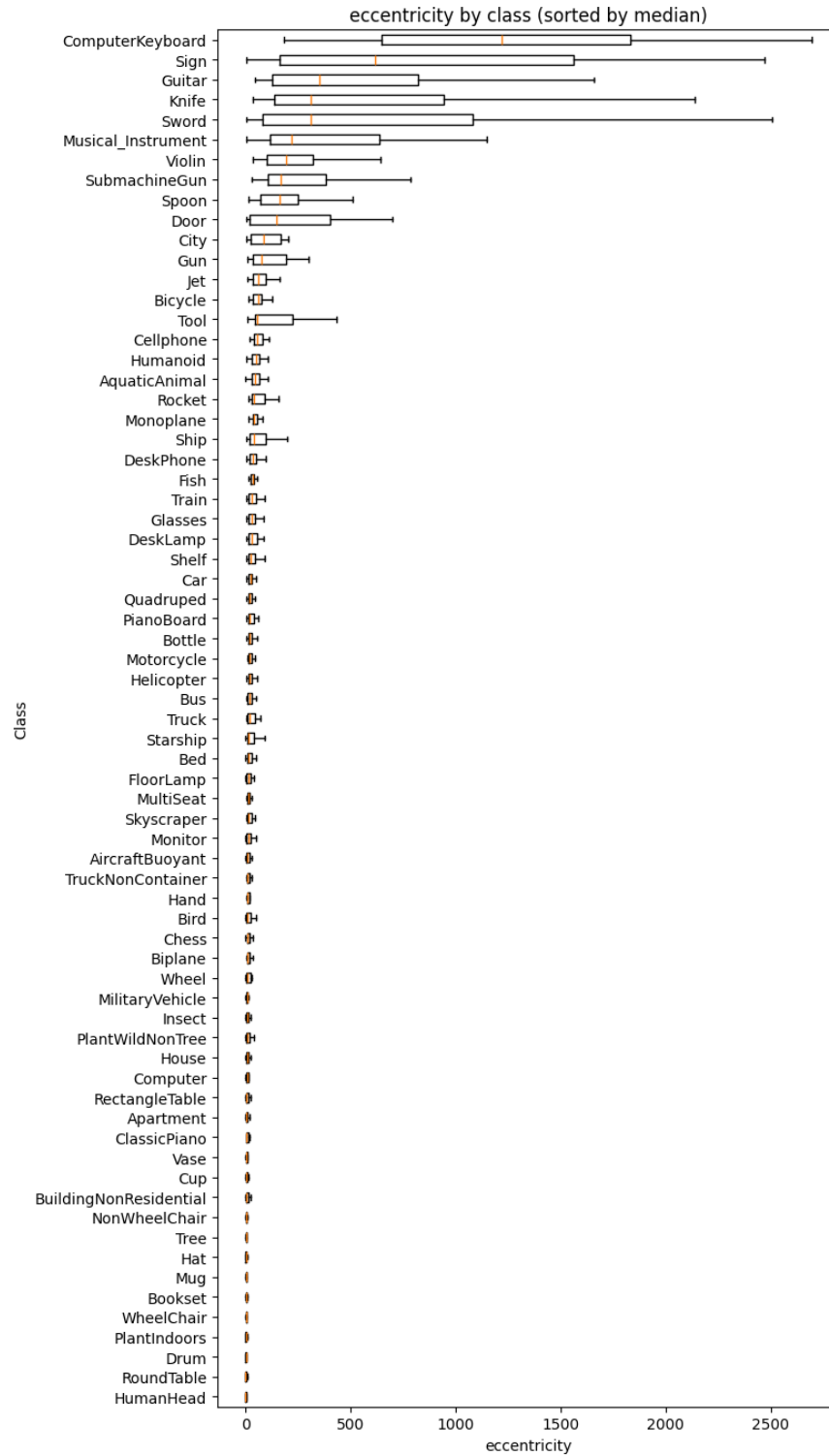


Figure 39: **Boxplot of *Eccentricity* values by class (sorted by median)**. Eccentricity reflects elongation along principal axes. This plot makes it easy to compare which object classes are more isotropic versus more elongated.

7.2.3 Shape property descriptors

While the previously discussed global descriptors capture overall geometric properties of a mesh through single scalar values, they do not describe how these properties are distributed across the object. To address this limitation, we employ a set of statistical shape property descriptors: A3, D1, D2, D3, and D4, that capture local geometric relationships through large scale vertex sampling. Instead of returning a single measurement, each of these descriptors is represented as a normalized histogram that summarises the frequency of geometric relations such as angles, distances, triangle areas, or tetrahedron volumes. These descriptors provide a richer statistical characterization of shape structure by capturing not only the global extent but also the internal spatial variability of the geometry. Because they are based on random vertex sampling and operate on unified meshes, they are invariant to translation, rotation, and scale, and can effectively differentiate objects with similar overall size but distinct local geometric features.

7.2.3.1 A3: Angle distribution

The A3 descriptor characterizes the distribution of angles formed by triplets of randomly selected vertices. For $(\mathbf{x}, \mathbf{y}, \mathbf{z}) \in \mathbf{V}^3$, the angle at vertex $\mathbf{x}$ is computed as

$$\theta = \arccos\left(\frac{(\mathbf{y} - \mathbf{x}) \cdot (\mathbf{z} - \mathbf{x})}{\|\mathbf{y} - \mathbf{x}\| \|\mathbf{z} - \mathbf{x}\|}\right). \quad (36)$$

We perform this computation for a total of 100,000 samples, where each vertex is selected randomly with replacement. The resulting angle values are then aggregated into a frequency histogram. Before constructing the histogram, it was necessary to determine the appropriate range of the horizontal axis. The minimum possible angle was set to 0, while for the maximum we initially considered the theoretical upper bound $\theta = \pi$. However, using the theoretical limit resulted in many empty bins in several histograms. To obtain a more informative distribution, we instead determined the empirical upper bound from the dataset by computing the 99th percentile of all sampled angles and extending this value by an additional 10% buffer. In practice, this adjusted upper limit was still close to the theoretical bound, at approximately $\theta \approx \pi$.

Finally, the histogram was divided into 20 bins, which provided a good balance between descriptive granularity and computational efficiency.

To verify that the computed A3 descriptor values were reasonable and informative, we analyzed the consistency of the resulting distributions both *within* and *between* object classes. Specifically, we plotted the per-shape angle distributions for 20 representative classes, selected to be near the median in terms of the number of available shapes per class, ensuring a fair comparison across categories. The resulting per-class distributions are shown in Figure 40.

Descriptor A3 — per-class distributions (each line = shape)

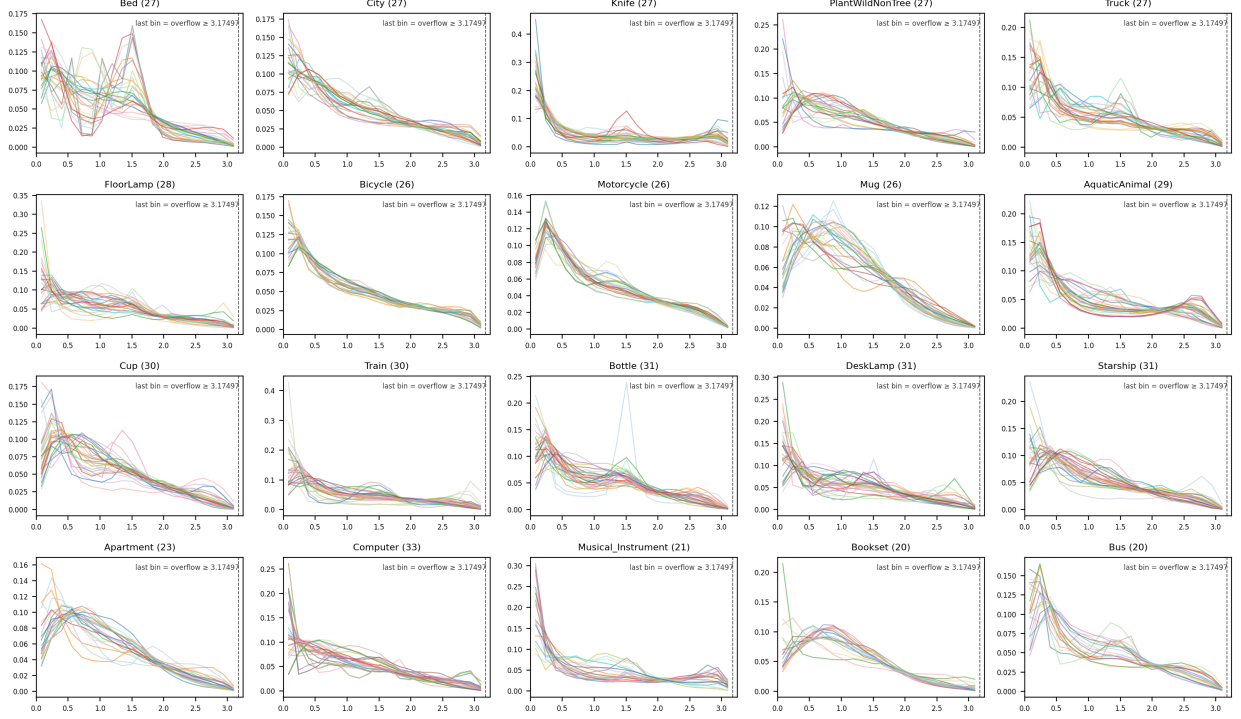


Figure 40: **Per-class A3 distributions (each line represents one shape)**. The plots show the frequency distributions of sampled inter-vertex angles for 20 object classes. For each class, the shapes exhibit relatively consistent intra-class distributions, while the overall profiles differ noticeably between classes. This indicates that the A3 descriptor captures geometric characteristics that are both stable within a class and discriminative across different object categories.

From these results, several observations can be made. First, despite some variability and occasional outliers, shapes within the same class tend to follow a similar distribution, suggesting that the descriptor is stable under intra-class variation. Second, the distributions across different classes vary substantially in both shape and relative peak positions, supporting the discriminative potential of the A3 descriptor for class-level geometry analysis. Overall, these findings confirm that A3 provides a meaningful statistical representation of angular structure across 3D objects.

7.2.3.2 D1: Distance from barycenter to surface points

The D1 descriptor quantifies how far the surface vertices of a shape lie from its mass barycenter. It is defined as the Euclidean distance between each sampled vertex $\mathbf{x}$ and the barycenter $\mathbf{b}(M)$:

$$d_1 = \|\mathbf{x} - \mathbf{b}(M)\|_2. \quad (37)$$

In the implementation, all mesh vertices are used directly, corresponding to $n = |\mathbf{V}|$ iterations per mesh. For each vertex, this distance is computed and then grouped into a 20-bin histogram spanning from 0 to the empirically determined upper bound. The resulting distribution describes how evenly mass is distributed around the barycenter.

Per-class histograms (Figure 41) reveal that D1 captures differences in compactness: spherical or cubic shapes exhibit narrow, symmetric distributions around their mean distance, whereas elongated or branched shapes show flatter and wider profiles.

Descriptor D1 — per-class distributions (each line = shape)

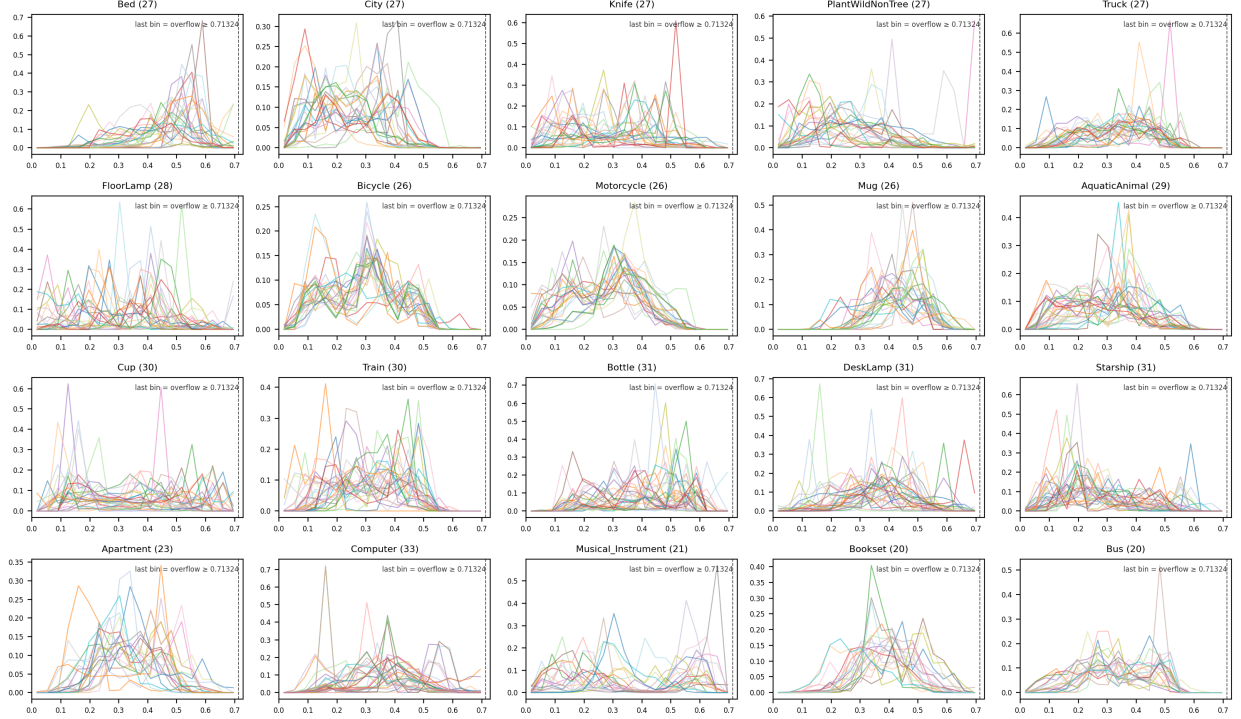


Figure 41: **Per-class D1 distributions (each line represents one shape).** The distributions show distances from the barycenter to sampled vertices across 20 object classes. Compact objects show narrow peaks, while extended or articulated ones have broader spreads, demonstrating that D1 effectively captures spatial dispersion.

While the general shapes of the D1 distributions differ noticeably between classes indicating that the descriptor successfully captures key geometric characteristics, there is also substantial variation within individual classes. This intra-class variability introduces a notable amount of noise into the distributions, which can lower the similarity scores among shapes belonging to the same category. A possible contributing factor is the remeshing process: uneven vertex sampling or local vertex density variations can significantly influence the computed distances from the barycenter, thereby affecting the resulting histogram shape.

7.2.3.3 D2: Distance between two random vertices

The D2 descriptor characterises the distribution of distances between pairs of randomly selected vertices. For $(\mathbf{x}, \mathbf{y}) \in \mathbf{V}^2$, the distance is computed as

$$d_2 = \|\mathbf{x} - \mathbf{y}\|_2. \quad (38)$$

In code, 100,000 random pairs are generated using replacement, ensuring a diverse sampling even for meshes with fewer vertices. Each distance value is added to a 20-bin histogram within the range $[0, d_{\max}]$, where

$d_{\max}$ was determined globally via the 99th percentile plus 10% buffer.

The resulting distributions (Figure 42) reveal differences in spatial scale between object classes. Compact shapes concentrate distances near zero, while large or elongated shapes produce wider histograms extending toward the upper range. D2 thus provides a global measure of spatial extent and object elongation, complementing D1 by comparing internal pairwise relationships rather than distances to the center.

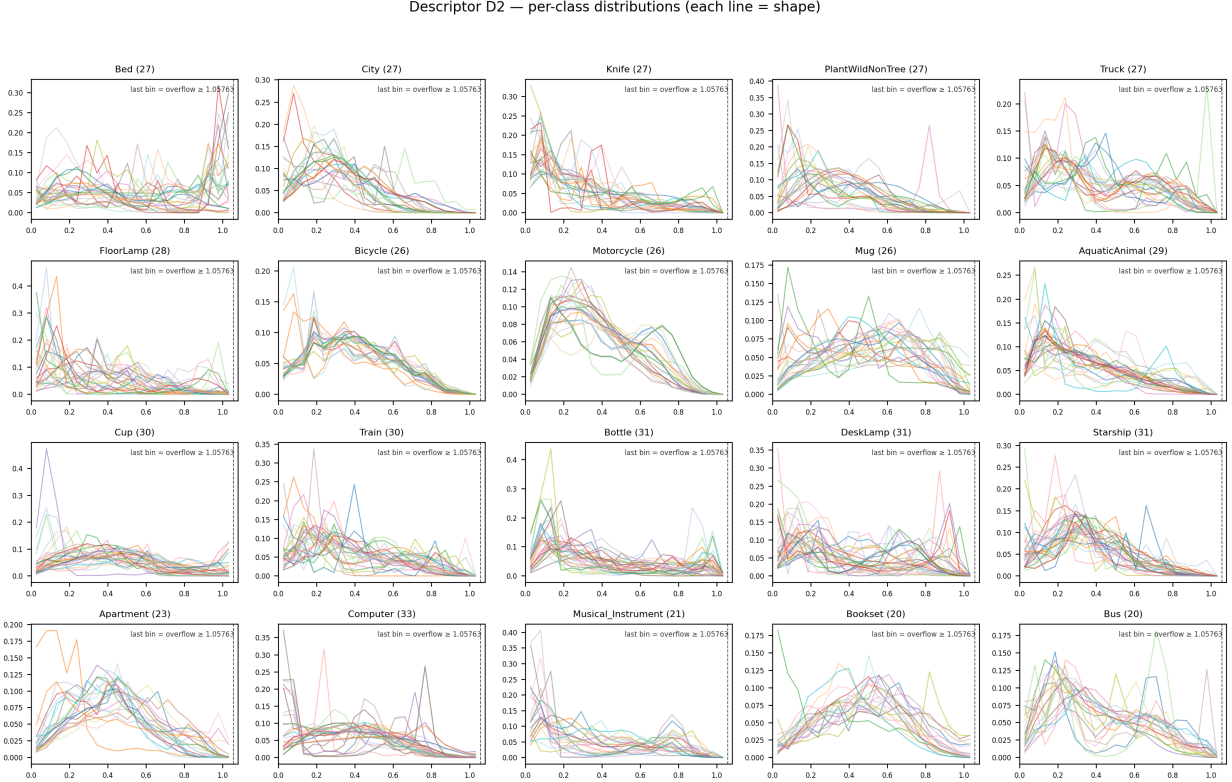


Figure 42: **Per-class D2 distributions (each line represents one shape)**. The plots show the distribution of pairwise vertex distances across 20 representative object classes. Compact geometries show sharp peaks at shorter distances, whereas spread-out structures exhibit flatter profiles across a broader range.

We can observe clear trends within specific classes. For example, the *AquaticAnimal* class tends to exhibit an early peak in its angle distribution, whereas the *Cup* class shows a more even spread across the entire range. Although a few outliers remain, this is expected, as objects within a single class naturally vary in shape and structure. Furthermore, the noticeable differences between classes reinforce that the A3 descriptor effectively captures distinctive geometric characteristics, providing the desired discriminative power for shape comparison.

7.2.3.4 D3: Square root of area of triangle given by three random vertices

The D3 descriptor extends the concept of geometric sampling to areas of triangles defined by three randomly chosen vertices. For $(\mathbf{x}, \mathbf{y}, \mathbf{z}) \in \mathbf{V}^3$, the triangle area is computed as

$$A_{\Delta} = \frac{1}{2} \|(\mathbf{y} - \mathbf{x}) \times (\mathbf{z} - \mathbf{x})\|. \quad (39)$$

In the implementation, 100,000 random triplets of vertices are sampled with replacement. Each triangle’s area is then square-rooted before histogramming to express it in length units and mitigate the effect of large outliers. The histogram again uses 20 bins over the range $[0, a_{\max}]$, with $a_{\max}$ empirically determined via the global 99th percentile plus 10% margin.

Figure 43 shows per class D3 distributions, which reveal meaningful variation across object categories. Flat, planar shapes yield histograms skewed toward higher area values, while curved or finely detailed surfaces produce peaks near zero.

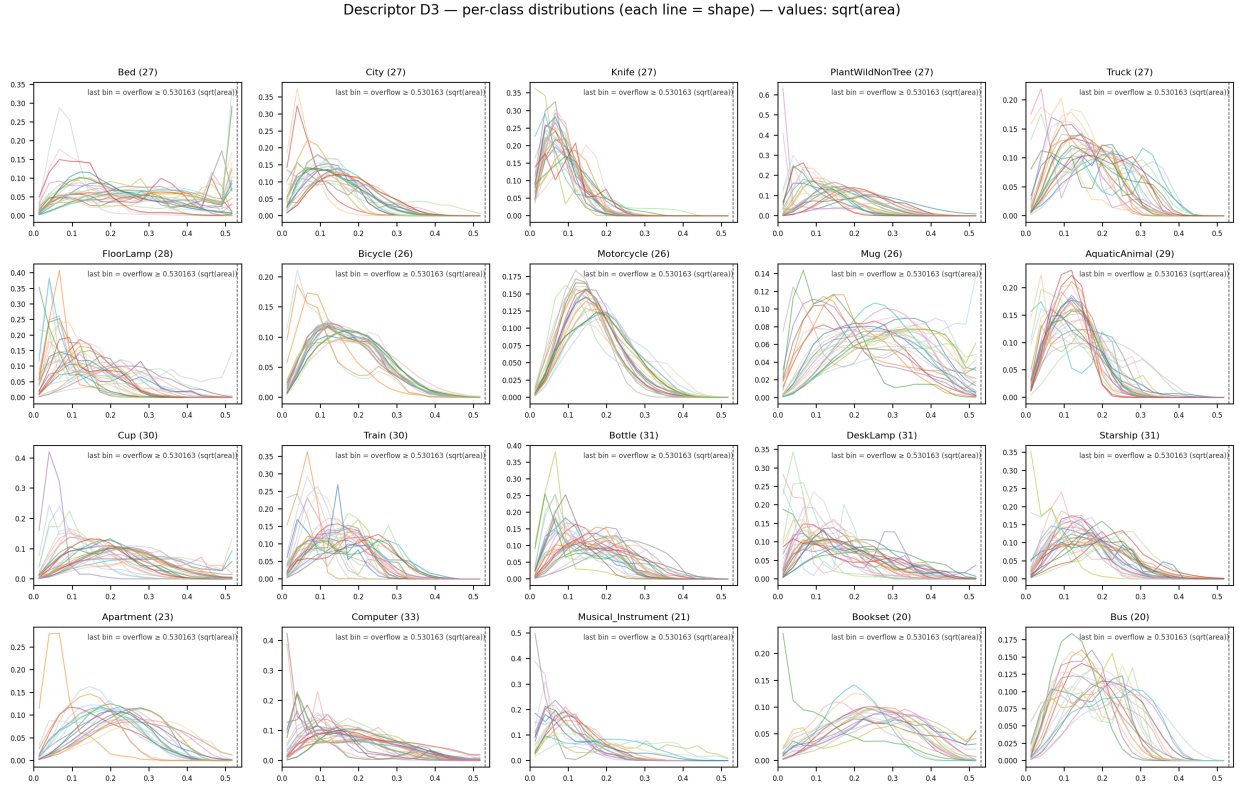


Figure 43: **Per-class D3 distributions (each line represents one shape).** The plots show the frequency of sampled triangle areas (square-rooted) across 20 object classes. Planar shapes show higher area peaks, while intricate or curved surfaces produce low area concentrations.

Therefore, D3 captures the typical scale and variability of planar patches within a mesh, serving as a proxy for surface complexity and curvature structure.

Within most classes, we again observe consistent trends across individual shapes, indicating that the descriptor captures meaningful structural information. For instance, elongated objects such as *Bicycles* or *Knives* tend to show a concentration of smaller area values, while more voluminous objects such as *Cups* or *Bottles* exhibit a broader spread. As with the other descriptors, some variation and outliers remain, which is expected due to the natural diversity within each object class. The differences between class-level distributions again demonstrate that the D3 descriptor provides strong discriminative power, further supporting its suitability for shape comparison and retrieval tasks.

7.2.3.5 D4: Cube root of volume of tetrahedron formed by four random vertices

The D4 descriptor generalises the previous sampling strategy to four vertices, measuring the volumes of tetrahedra they define. For $(\mathbf{w}, \mathbf{x}, \mathbf{y}, \mathbf{z}) \in \mathbf{V}^4$, the tetrahedron volume is computed as

$$V_{\text{tet}} = \frac{1}{6} |(\mathbf{x} - \mathbf{w}) \cdot ((\mathbf{y} - \mathbf{w}) \times (\mathbf{z} - \mathbf{w}))|. \quad (40)$$

As in the implementation, 100,000 random quadruples are sampled with replacement. The cube root of each volume is taken before histogramming to express values in consistent length units and reduce skew caused by large outliers. All values are grouped into a 20-bin histogram over $[0, v_{\text{max}}]$, where v_{max} is set to the global 99th percentile plus 10% buffer.

Per-class D4 histograms (Figure 44) reveal distinctive volumetric signatures: solid, compact shapes show broader, higher volume distributions, whereas thin or planar structures concentrate near zero.

Descriptor D4 — per-class distributions (each line = shape) — values: $\text{cb}(\text{volume})$

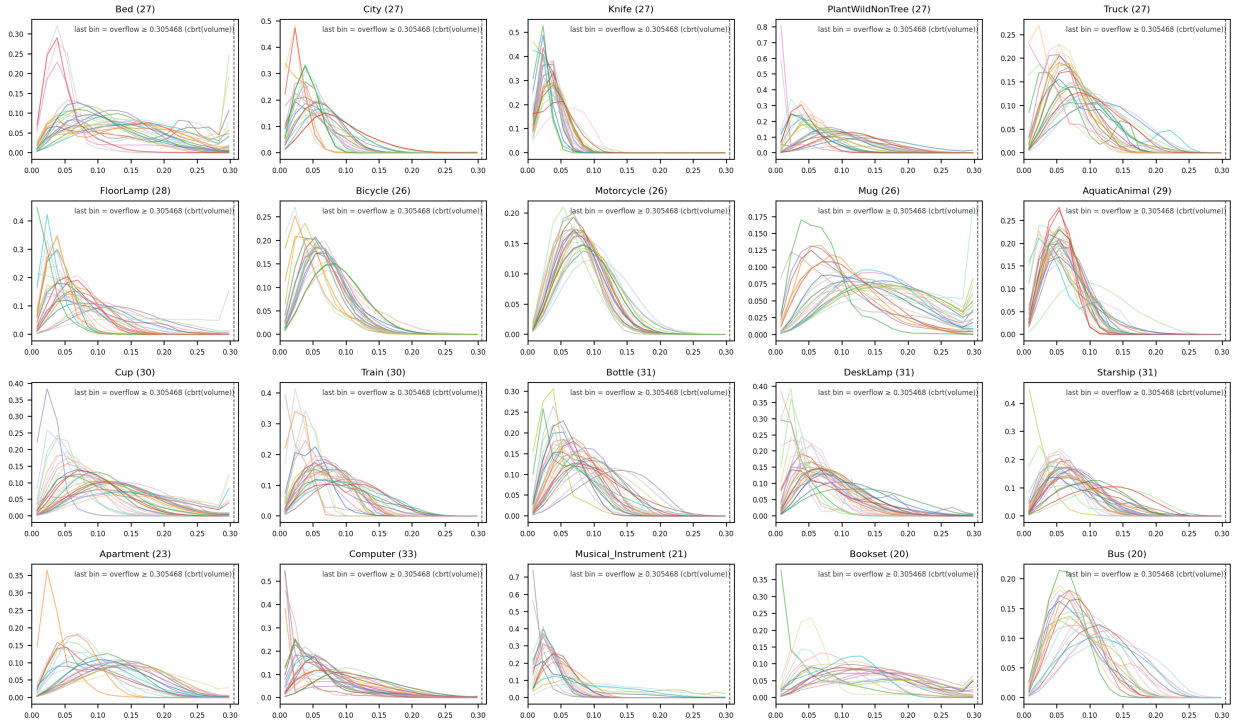


Figure 44: **Per-class D4 distributions (each line represents one shape)**. The plots display distributions of tetrahedron volumes (cube-rooted) sampled across 20 representative object classes. Higher values indicate volumetrically dense shapes, while lower values reflect thin, sheet-like geometries.

The D4 descriptor captures volumetric complexity and internal density, complementing the distance based metrics by describing how the object fills three-dimensional space. With a few outliers aside the histograms look very similar within the same class. However, the distance in between the classes is also small which might reduce the effectiveness of this descriptor.

- 8 Step 4: Querying
- 9 Step 5: Scalability
- 10 Step 6: Evaluation

References

- [1] Philip Shilane et al. “The Princeton Shape Benchmark”. In: *Proceedings of the Shape Modeling International Conference*. IEEE, 2004, pp. 167–178. DOI: [10.1109/SMI.2004.1314504](https://doi.org/10.1109/SMI.2004.1314504). URL: <https://shape.cs.princeton.edu/benchmark/>.
- [2] Paul Bourke. *Object Files (.obj)*. <http://paulbourke.net/dataformats/obj/>. Accessed: December 2024. 1999.
- [3] Plotly Technologies Inc. *Dash: A web application framework for Python*. <https://plotly.com/dash>. Version 2.0 and above. 2017.
- [4] Plotly Technologies Inc. *Plotly.py: An interactive, open-source, and browser-based graphing library for Python*. <https://plotly.com/python>. Version 5.0 and above. 2015.
- [5] Wes McKinney et al. “pandas: a foundational Python library for data analysis and statistics”. In: *Python for high performance scientific computing*. Vol. 14. 9. Austin, TX. 2011, pp. 1–9.
- [6] Charles R. Harris et al. “Array programming with NumPy”. In: *Nature* 585.7825 (2020), pp. 357–362. DOI: [10.1038/s41586-020-2649-2](https://doi.org/10.1038/s41586-020-2649-2).
- [7] John Deacon. “Model-view-controller (mvc) architecture”. In: *Online* [Citado em: 10 de março de 2006.] <http://www.jdl.co.uk/briefings/MVC.pdf> 28 (2009), p. 61.
- [8] Rufu Qin et al. “Web-based 3D visualization framework for time-varying and large-volume oceanic forecasting data using open-source technologies”. In: *Environmental Modelling & Software* 135 (2021), p. 104908.
- [9] Plotly Technologies Inc. *Plotly Graph Objects Python Library*. 2025. URL: <https://plotly.com/python/graph-objects/>.
- [10] Mario Botsch et al. *Polygon Mesh Processing*. CRC Press, 2010. URL: <https://www.pmp-book.org/>.
- [11] Qian-Yi Zhou, Jaesik Park, and Vladlen Koltun. “Open3D: A Modern Library for 3D Data Processing”. In: *arXiv:1801.09847* (2018).
- [12] Andrzej Maćkiewicz and Waldemar Ratajczak. “Principal components analysis (PCA)”. In: *Computers & Geosciences* 19.3 (1993), pp. 303–342.
- [13] Fabian Pedregosa et al. “Scikit-learn: Machine learning in Python”. In: *Journal of machine learning research* 12 (2011), pp. 2825–2830.